



UNIVERSITY OF BARISHAL

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Software Requirements Specification(SRS)

CCTV Based Rescue &Emergency Detection System

CSE-3104 : Software Engineering and information System Design Lab.

Submitted By :

Team Return0

Abdullah Al Maruf (230102023)

Sadia Mostarin Shuvra (230102032)

Ahasanul Islam Mugdho (230102025)

Deep Paul (230102018)

Antor Sarker (230102021)

Dip Dey (230102038)

Submitted To :

Md Samsuddoha

Assistant Professor, Dept. of CSE, University of Barishal

Date of Submission : 14 June

Table of Contents

1. Inception -----	
1.1 Introduction -----	
1.2 Identifying Stakeholders -----	
1.3 Recognizing Multiple Viewpoints -----	
1.4 Working Towards Collaboration -----	
1.5 Asking the First Questions -----	
2. Elicitation -----	
2.1 Introduction -----	
2.2 Eliciting Requirements -----	
2.3 Collaborative Requirements Gathering -----	
2.4 Quality Function Deployment -----	
2.5 Usage Scenarios -----	
2.6 Elicitation Work Products -----	
3. User Stories -----	
3.1 User Stories – CCTV Operator -----	
3.2 User Stories – Emergency Responder -----	
3.3 User Stories – System Administrator -----	
3.4 User Stories – Municipal Authority -----	
3.5 User Stories – General Public -----	
4. Use Case Scenarios -----	
5. Use Case Descriptions -----	
5.1 Camera Management Use Cases -----	

5.2	Detection Engine Use Cases -----
5.3	Alert Management Use Cases -----
5.4	Incident Management Use Cases -----
5.5	Dashboard & Analytics Use Cases -----
5.6	User & Role Management Use Cases -----
6.	Use Case Diagrams -----
7.	Activity & Swimlane Diagrams -----
7.1	Fire Detection and Alert Flow -----
7.2	Accident Detection and Response Flow -----
7.3	Operator Verification Flow -----
7.4	Admin Camera Management Flow -----
8.	Entity Relationship (ER) Diagram -----
8.1	Entity Identification -----
8.2	Attributes and Primary Keys -----
8.3	Relationships and Cardinalities -----
8.4	Complete ER Diagram -----
9.	Class-Based Modeling -----
9.1	Class Identification -----
9.2	Detailed Class Attributes and Operations -----
9.3	Class Relationships -----
9.4	Class Diagram -----
10.	Behavioral Modeling -----
10.1	Event Table -----
10.2	State Transition Diagrams -----

11. Data Flow Diagrams (DFD) -----
11.1 Level 0 – Context Diagram -----
11.2 Level 1 – Main Processes -----
11.3 Level 2 – Detailed DFDs -----
12. Sequence Diagrams -----
12.1 Fire Detection and Alert Sequence -----
12.2 Accident Detection and Dispatch Sequence -----
12.3 Operator Verification Sequence -----
12.4 Admin Camera Registration Sequence -----
13. Non-Functional Requirements -----
14. Constraints & Assumptions -----
15. Conclusion -----

CCTV-Based Rescue and Emergency Detection System

CCTV-Based Rescue and Emergency Detection System is an intelligent, AI-powered solution that automatically detects **fire incidents** and **road accidents** from live CCTV video streams. It focuses on automatically identifying two critical emergency events from live CCTV feeds: **fire incidents** and **road accidents**. Traditional surveillance requires human operators to constantly watch multiple screens, which often leads to delayed detection and slower emergency response. This AI-powered solution continuously analyzes video streams using computer vision models. For fire, the system detects flames, smoke patterns, and temperature anomalies. For accidents, it identifies vehicle collisions, sudden stops, rollovers, and even bike or pedestrian impacts. Once an incident is detected, the system immediately classifies its severity (low, medium, high, or critical) and creates a time-stamped record with a short video clip.

The system then triggers a multi-channel alert mechanism. CCTV operators receive visual and audio notifications on their dashboard, allowing them to quickly verify or reject the alert as a false alarm. Simultaneously, emergency services—fire brigade, ambulance, and police—are automatically dispatched with precise GPS location, severity level, and a live video feed link. This automated process eliminates the delay caused by manual reporting and human error, ensuring that rescue teams are informed within seconds of an incident occurring. Operators can also manually report any missed event, and all false alarms are logged to improve the AI's accuracy over time.

Key features include camera registration with geolocation, adjustable detection sensitivity per camera, automatic video clip storage, a real-time operator dashboard, incident heatmaps for city-wide risk analysis, and detailed analytics on response time improvement. The system benefits the public through faster rescue, reduces the workload on CCTV operators, and gives municipal authorities data-driven insights for better urban safety planning. By turning passive

surveillance into an active life-saving tool, this project directly addresses the critical gap between an emergency occurring and help arriving on the scene.

1. Inception

1.1 Introduction

Inception is the beginning phase of requirements engineering. It defines how a software project gets started and what is the scope and nature of the problem to be solved. The goal of the inception phase is to identify concurrence needs and conflict requirements among the stakeholders of a software project. For the CCTV-Based Emergency and Rescue Detection System, the inception phase is critical because the system deals with life-saving scenarios where every second counts.

The traditional approach to emergency detection relies on human monitoring of CCTV cameras. A single security officer may need to watch dozens of video feeds simultaneously. Studies show that after 20 minutes of continuous monitoring, a human operator misses up to 45% of critical events. This leads to delayed emergency response, which directly increases casualties and property damage. For example, a fire that could have been contained in 2 minutes may spread uncontrollably if detected 5 minutes late. Similarly, road accident victims lose critical "golden hour" treatment opportunities when ambulances are dispatched late.

Our proposed system addresses these limitations by introducing an artificial intelligence layer that continuously analyzes video streams in real-time. The AI models are specifically trained to detect two high-impact emergency categories: fire incidents (flames, smoke) and road accidents (collisions, rollovers, bike accidents, pedestrian impacts). Once detected, the system automatically classifies the severity, records a video clip, and triggers multi-channel alerts. This reduces

detection time from minutes (human response) to milliseconds (machine response).

To establish the groundwork for this system, we have worked with the following factors related to the inception phases:

1. Identifying Stakeholders – Everyone affected by or affecting the system.
2. Recognizing multiple viewpoints – Understanding diverse expectations.
3. Working towards collaboration – Resolving conflicts through discussion.
4. Asking the First Questions – Clarifying scope, goals, and constraints.

1.2 Identifying Stakeholders

A stakeholder refers to any individual or group who is directly or indirectly affected by the development, deployment, and operation of the system. Stakeholders include users who interact with the system regularly, as well as organizations and individuals who influence system decisions, maintenance, and policy enforcement.

To identify the stakeholders of the CCTV-Based Emergency & Rescue Detection System, we analyzed the objectives of the platform and considered the following questions:

- Who will use the system regularly? (CCTV operators, administrators)
- Who will manage and control system operations? (Admins, municipal IT departments)

- Who will benefit from the outcomes of the system? (General public, emergency services, city authorities)
- Who will support financial transactions and technical maintenance? (Municipal budget, technical maintenance team)
- Whose activities will be affected after system deployment? (Emergency dispatchers, traffic police, hospital triage units)

1.2.1 General Public (Citizens)

The general public is the primary beneficiary of the system. They do not directly interact with the detection software but benefit from faster emergency response times. When a fire breaks out in a residential building or a serious accident occurs on a highway, the system ensures that help arrives minutes earlier. This can mean the difference between life and death. Additionally, public awareness of the system may encourage safer driving behavior. Citizens also have legitimate privacy concerns that must be addressed through privacy masking and data protection policies.

1.2.2 CCTV Operators (Security Personnel)

CCTV operators are the primary human users of the system. They work in control rooms monitoring dozens or hundreds of cameras. Their daily challenge is fatigue and attention decay. Our system assists them by acting as an automated watchdog that never tires. Operators receive clear, prioritized alerts only when something important happens. They then verify the alert (confirm or mark as false alarm) and coordinate with emergency services. Their responsibilities include:

- Monitoring the operator dashboard
- Verifying AI-generated alerts within seconds
- Manually reporting incidents that AI might miss

- Adjusting camera views when needed
- Escalating critical incidents to supervisors

1.2.3 Emergency Response Teams

This group includes fire brigade personnel, ambulance paramedics, and police officers. They receive real-time notifications from the system containing:

- Exact GPS coordinates of the incident
- Severity classification (e.g., small fire vs. large fire; minor fender-bender vs. multi-vehicle pileup)
- Live video feed link to assess the scene before arrival
- Estimated number of victims (for accidents)
- Nearby hydrant locations (for fires)

With this information, responders can prepare appropriate equipment, choose the fastest route, and even request additional units before reaching the scene. This dramatically improves response effectiveness.

1.2.4 System Administrator

The system administrator is responsible for managing the overall platform. This role is typically held by a senior IT staff member in the municipal corporation or a contracted service provider. Responsibilities include:

- Registering new cameras with their locations and stream URLs
- Adjusting detection sensitivity per camera zone (e.g., higher sensitivity in high-risk areas)

- Managing user accounts and role-based permissions
- Monitoring system health and performance
- Generating incident reports for authorities
- Reviewing audit logs for compliance and troubleshooting
- Coordinating with the technical maintenance team for updates

1.2.5 Municipal Authorities (City Corporation)

Municipal authorities are the decision-makers who approve budgets, oversee deployment, and use system analytics for urban planning. They are interested in:

- City-wide incident statistics and heatmaps
- Reduction in emergency response times (key performance indicator)
- Return on investment (lives saved vs. system cost)
- Integration with other city systems (traffic management, public transport)
- Public safety improvements that can be highlighted to citizens

1.2.6 Technical Maintenance Team

This team consists of software developers, AI engineers, and hardware technicians. They are responsible for:

- Designing, implementing, and maintaining the detection algorithms
- Updating AI models with new training data (to reduce false positives)
- Ensuring system scalability (adding more cameras without performance loss)
- Debugging issues reported by operators or admins
- Managing server infrastructure (on-premise or cloud)
- Handling database backups and disaster recovery

1.2.7 Cloud / Storage Service Provider

If the system is deployed in the cloud (or uses cloud storage for video clips), the provider becomes an indirect stakeholder. They ensure that video footage, detection logs, and incident records are securely stored and retrievable when needed. Service Level Agreements (SLAs) must define uptime, data retention policies, and privacy compliance.

1.2.8 Insurance Companies

Insurance companies may use system data to verify claims. For example, a fire incident recorded by the system provides objective evidence for claim processing. Similarly, accident footage can help determine fault. This stakeholder benefits from reliable, timestamped, and unaltered video evidence.

1.3 Recognizing Multiple Viewpoints

Each stakeholder group has different expectations and concerns regarding the CCTV Emergency Detection System. Recognizing these viewpoints helps in resolving conflicts and aligning system requirements with stakeholder needs. The following subsections detail each viewpoint.

1.3.1 General Public Viewpoints

Faster emergency response: The primary expectation is that the system will reduce the time between incident occurrence and rescue arrival.

Reliable detection: Citizens expect the system to detect genuine emergencies accurately and not generate excessive false alarms that desensitize responders.

Privacy protection: Surveillance should not intrude into private spaces (homes, changing rooms). Privacy zones must be masked.

Transparency: People want to know where cameras are located and how footage is used.

Accountability: If the system fails to detect a major incident, citizens expect an explanation and remedy.

1.3.2 CCTV Operator Viewpoints

Clear and non-intrusive alerts: Alerts should be noticeable but not overwhelming. A dedicated alert panel with sound is preferred.

Quick verification workflow: The interface should allow confirming or rejecting an alert with minimal clicks (ideally one or two).

Ability to handle false positives: Operators need a simple way to mark false alarms and optionally provide feedback (e.g., "tree shadow mistaken for smoke").

Dashboard simplicity: With many cameras, the dashboard must be organized. Thumbnails, camera groups (by zone), and search/filter functions are essential.

Manual override: Operators must be able to manually create an incident report if they spot something the AI missed.

Fatigue reduction: The system should reduce, not increase, operator workload. Poorly designed alert storms would be counterproductive.

1.3.3 Emergency Response Team Viewpoints

Real-time location data: GPS coordinates must be accurate. Integration with navigation apps (Google Maps, Waze) is highly desired.

Severity estimation: Responders need to know if it's a small trash fire or a large building fire; a minor side-swipe or a multi-car pileup with trapped victims.

Live video feed access: Viewing the incident before arrival allows pre-planning. For example, firefighters can see which side of a building is most affected; paramedics can spot the number of injured.

Integration with dispatch systems: The system should push data directly into Computer-Aided Dispatch (CAD) systems to avoid manual re-entry.

Response confirmation: Responders want to mark that they have acknowledged and are en route, and later mark the incident as resolved.

1.3.4 System Administrator Viewpoints

Full administrative control: Ability to add, modify, or remove cameras, users, and system settings.

Adjustable detection sensitivity: Different zones may need different thresholds. A busy intersection might require higher confidence for alerts to avoid false alarms from near-misses.

Comprehensive reporting: Generate reports by date, camera, incident type, severity, verification status, and response times.

Audit logs: All user actions (login, verification, sensitivity changes) must be logged for security and troubleshooting.

Role-based access control: Ensure operators cannot change admin settings; emergency responders have read-only access to incident data.

System health monitoring: Alerts when a camera goes offline, storage is full, or detection service crashes.

1.3.5 Municipal Authority Viewpoints

City-wide incident heatmaps: Visualize where fires and accidents occur most frequently. This guides resource allocation (e.g., adding more fire stations or traffic lights).

Performance metrics: Measure reduction in emergency response time after system deployment. This justifies the budget.

Public safety ROI: Statistics on lives saved, property damage reduced, and emergency resource optimization.

Integration with existing infrastructure: The system should complement, not replace, existing emergency communication networks.

1.3.6 Technical Team Viewpoints

Easy to develop and maintain: Clean code, modular architecture, and thorough documentation.

No ambiguous requirements: Each requirement must be testable and unambiguous.

Scalable architecture: The system should handle hundreds or thousands of cameras without redesign.

Robust logging: Detailed logs for debugging false detections and system errors.

Model update mechanism: Ability to retrain and deploy AI models without downtime.

1.4 Working Towards Collaboration

Since stakeholders may have overlapping or conflicting requirements, collaboration is necessary to reach a balanced and practical solution. To achieve this, the following steps were followed:

1. Identification of common and conflicting requirements –

We listed all requirements from each stakeholder group and identified overlaps (e.g., everyone wants accurate detection) and conflicts (e.g., high sensitivity vs. low false positives).

2. Categorization of requirements based on priority –

Using MoSCoW method (Must have, Should have, Could have, Won't have).

3. Evaluation of feasibility and system constraints –

Technical limits (processing power, storage, network bandwidth) and budget constraints were considered.

4. Finalization of requirements through consensus –

Meetings were held with representative stakeholders to negotiate trade-offs.

1.4.1 Common Requirements

All stakeholder groups agreed on the following:

Real-time detection accuracy– The system must detect genuine emergencies with high accuracy.

Low false positive rate – Too many false alarms will cause operators and responders to ignore alerts.

Reliable alerting mechanism – Alerts must be delivered even during network fluctuations.

Secure and role-based access – Unauthorized access to video feeds or incident data is unacceptable.

Web-based dashboard access – Operators and admins prefer browser-based interfaces accessible from control room computers.

1.4.2 Conflicting Requirements

The following conflicts were identified and resolved through prioritization:

Conflict	Stakeholder A	Stakeholder B	Resolution
Maximum detection accuracy vs. processing cost	Public (wants perfect detection)	Municipal budget (limited funds)	Accept 95% accuracy with human verification; upgrade hardware as budget allows.
Continuous recording vs. storage limits	Operators (want full footage)	IT budget (limited storage)	Store only triggered clips (30 seconds before and after incident) plus continuous low-res for 7 days.
High sensitivity vs. false alarm rate	Fire brigade (want no missed fires)	Operators (annoyed by false alarms)	Adjustable sensitivity per camera; high-sensitivity zones near high-risk areas only.
Privacy masking vs. detection coverage	General public (privacy)	Police (full coverage)	Mask private property (homes, schools) but keep public roads fully visible.

1.4.3 Final Requirements (Consensus)

After analysis and prioritization, the following final requirements were agreed upon:

FR-01 : Real-time fire detection using CCTV video feeds (flame and smoke recognition)

FR-02 : Real-time road accident detection (vehicle collisions, rollovers, bike accidents, pedestrian impacts)

FR-03 : Automated alerts to emergency services (fire, ambulance, police) with GPS and severity

- FR-04 : Dashboard for CCTV operators with live video feeds and alert queue
- FR-05 : Incident verification mechanism (operator confirms or marks false alarm)
- FR-06 : Geo-location tagging for each camera (GPS coordinates)
- FR-07 : Severity classification for fire: Small, Medium, Large, Critical
- FR-08 : Severity classification for accidents: Minor, Major, Critical
- FR-09 : Video recording and storage of incidents (10 seconds before + 30 seconds after)
- FR-10 : Admin panel for system configuration (camera registration, sensitivity, user roles)
- FR-11 : Incident reporting and analytics dashboard (heatmaps, statistics, exports)
- FR-12 : Privacy masking for non-public areas (configurable per camera)
- FR-13 : Manual incident reporting by operators (if AI misses)
- FR-14 : Real-time notification to responder mobile apps (iOS/Android)
- FR-15 : Integration with existing CAD (Computer-Aided Dispatch) systems

1.5 Asking the First Questions

During the inception phase, a set of context-free questions was used to understand the project goals, stakeholder expectations, and system constraints. These questions helped clarify:

Question & Answer

1. What is the core problem to be solved?

-> Delayed emergency response due to manual CCTV monitoring limitations.

2. Who is behind this idea?

-> A student team under university supervision, supported by municipal authorities.

3. Who will use the system?

-> CCTV operators, emergency responders, system administrators, municipal authorities.

4. What are the expected benefits?

-> Reduced response time (from minutes to seconds for detection), saved lives, reduced property damage.

5. What are the possible alternatives?

-> Manual monitoring, public reporting hotlines, automated smoke detectors (limited range).

6. How will success be measured?

-> Average detection-to-alert time, false positive rate, emergency response time improvement, user satisfaction.

7. What are the major constraints?

-> Budget for GPUs/servers, availability of CCTV infrastructure, network bandwidth, privacy regulations.

8. What happens if the system fails?

-> Manual fallback (operators can still monitor directly); redundant servers. |

The answers to these questions guided the requirement analysis and shaped the overall system vision.

1.6 Conclusion of Inception Phase

The inception phase of the CCTV Emergency Detection System established a clear understanding of the system's purpose, scope, and stakeholders. It helped identify key requirements, resolve conflicting viewpoints, and define a shared vision for the platform. This phase laid a strong foundation for subsequent requirement analysis, design, and development activities.

2. Elicitation

2.1 Introduction

Elicitation is a task that helps stakeholders define what is required from a software system. During the elicitation phase, several challenges may arise, such as problems of scope, problems of requirement volatility, and problems of understanding among stakeholders. Identifying and managing these issues at an early stage is essential to ensure that the system meets real user needs.

However, elicitation is not an easy task. To overcome these challenges, we conducted the requirements elicitation activities for the CCTV Emergency Detection System in a structured, organized, and systematic manner. This approach helped us clearly understand stakeholder expectations and transform them into well-defined system requirements.

The main challenges we faced during elicitation were:

Scope problems: Some stakeholders wanted additional features (e.g., gunshot detection, crowd panic detection). We limited scope to fire and accident detection for this version.

Volatility: Emergency response protocols changed during the project. We established a change control board.

Understanding gaps: Technical terminology (e.g., "confidence threshold") was unfamiliar to non-technical stakeholders. We used visual examples.

2.2 Eliciting Requirements

Unlike the inception phase, where a Question and Answer (Q&A) approach is primarily used, the elicitation phase employs a requirements elicitation format that combines elements of problem solving, elaboration, negotiation, and specification. This process requires close cooperation between end users, CCTV operators, emergency services, and developers.

To elicit requirements for the system, we carried out the following four activities:

1. Collaborative Requirements Gathering – Workshops and interviews.
2. Quality Function Deployment – Translating stakeholder "voices" into technical requirements.
3. Usage Scenarios – Walking through concrete examples of system use.
4. Elicitation Work Products – Documenting the results.

2.3 Collaborative Requirements Gathering

Various approaches to collaborative requirements gathering have been proposed, each utilizing different interaction scenarios. For the CCTV Emergency Detection project, we followed a collaborative approach involving discussions and analysis with potential stakeholders.

The following steps were completed during the collaborative requirements gathering process:

Step 1: Stakeholder Workshops

We conducted three half-day workshops with the following participants:

- 5 CCTV operators from the city control room
- 2 fire brigade officers
- 2 ambulance service dispatchers
- 1 municipal IT manager
- 2 system administrators (proposed)

Step 2: Interviews

Individual interviews were held with:

- A traffic police inspector (to understand accident response needs)
- A privacy law consultant (to ensure compliance with surveillance regulations)
- A hospital emergency department head (to understand information needs for triage)

Step 3: Observation Sessions

Team members observed CCTV operators during their shifts for 8 hours. We noted:

- How they currently detect incidents (scanning, zooming, playback)
- Typical response times (average 45 seconds to notice an incident, additional 90 seconds to verify and call)
- Common distractions and fatigue patterns

- Workarounds they developed (e.g., sticky notes on monitors)

Step 4: Requirement Refinement

Based on these discussions and observations, the initial list of 35 requirements was refined, merged, and prioritized down to 20 core functional requirements and 12 non-functional requirements.

2.4 Quality Function Deployment (QFD)

QFD translates stakeholder "voices" into technical requirements. We used a simplified QFD matrix.

2.4.1 Normal Requirements (Essential)

These are essential system functionalities. Absence of these would make the system unacceptable.

NR-01 : Real-time video stream ingestion

NR-02 : Fire detection (flame + smoke)

NR-03 : Accident detection (collision + rollover)

NR-04 : Alert generation

NR-05 : Camera registration and management

NR-06 : Incident logging

NR-07 : Role-based access control

NR-08 : Secure authentication

2.4.2 Expected Requirements

These improve user satisfaction proportionally—the better they work, the happier the users are.

ER-01: Severity classification

- Fire: Small (e.g., trash fire), Medium (room), Large (building), Critical (multiple buildings). Accident: Minor (fender bender), Major (multi-vehicle), Critical (with fire or entrapment).

ER-02 : Live feed access

- Operators and responders can view live stream of incident camera.

ER-03 : Incident verification

- One-click "Confirm" or "False Alarm" from alert panel.

ER-04 : False alarm reporting

- Optional text field to describe why (e.g., "shadow", "steam").

ER-05 : Push notifications

- To mobile apps of registered emergency responders.

ER-06 : Incident heatmap

- Color-coded map showing incident density over time.

ER-07 : Search and filter

- By date, camera, type, severity, verification status.

ER-08 : Video clip saving

- Save 10 seconds before + 30 seconds after detection.

2.4.3 Exciting Requirements

These are not expected by users but significantly enhance satisfaction and system uniqueness.

XR-01 :Smoke detection before visible flames

- Detect early-stage smoldering fires.

XR-02 :Automatic emergency call

- Direct API integration with 911/999 dispatch systems.

XR-03 : Traffic diversion suggestion

- Suggest alternate routes to avoid accident congestion.

XR-04 : Integration with traffic lights

- Turn traffic lights red near accident scene to clear path for ambulances.

2.5 Usage Scenarios

We developed three concrete usage scenarios to validate requirements.

Scenario 1: Fire Detection in a Shopping Mall

Context: 3 PM on a weekday. A small electrical fire starts behind a food court counter.

System action: Camera covering the area detects smoke within 8 seconds. Severity classified as "Small Fire". Alert generated.

Operator: Receives pop-up with video clip. Verifies in 5 seconds. Clicks "Confirm".

System: Sends notification to nearest fire station with GPS and live feed link. Outcome: Fire engine arrives in 4 minutes. Fire extinguished before spreading. No injuries.

Scenario 2: Highway Multi-Car Accident

Context: 9 PM, rainy night. A truck rear-ends a car, causing a 3-car pileup.

System action: Accident detection model identifies collision pattern. Severity: Major (multiple vehicles). Alert generated.

Operator: Verifies quickly. Marks "Confirmed – Multiple victims likely".

System: Notifies both ambulance and police. Sends live feed.

Outcome: Paramedics see on live feed that one car is smoking, so they request fire support preemptively.

2.6 Elicitation Work Products

The following documents were produced during elicitation:

1. Stakeholder interview transcripts
2. Observation logs
3. Prioritized requirements list (20 functional, 12 non-functional)
4. QFD matrix (mapping stakeholder voices to technical specs)
5. Glossary of terms (for non-technical stakeholders)
6. Risk register (identified 8 major risks, e.g., camera downtime, model drift)

3. User Stories

User stories are short, simple descriptions of a feature from the perspective of the end user. Below are detailed user stories for each actor.

3.1 User Stories – CCTV Operator

OP-01 : As a CCTV operator, I want to see live video feeds from all registered cameras on a single dashboard, so that I can monitor public areas efficiently without switching screens.

OP-02 : As a CCTV operator, I want to receive a visual (pop-up) and audio (beep) alert when fire or accident is detected, so that I can respond immediately even if I am looking at another camera.

OP-03 : As a CCTV operator, I want to verify an incident as "real" or "false alarm" with one click, so that emergency services are dispatched only for genuine emergencies.

OP-04 : As a CCTV operator, I want to view incident details including camera location, timestamp, severity classification, and a short video clip, so that I can provide accurate information to responders.

3.2 User Stories – Emergency Responder (Fire/Ambulance/Police)

ER-01 : As an emergency responder, I want to see the exact GPS location of the incident with a link to open in Google Maps or Waze, so that I can navigate accurately.

ER-02 : As an emergency responder, I want to access a live video feed of the incident before I arrive, so that I can assess the situation and prepare appropriate equipment.

ER-03 : As an emergency responder, I want to see severity classification (e.g., "Large Fire" or "Critical Accident"), so that I can send the right number of units.

ER-04 : As an emergency responder, I want to mark an incident as "En Route" when I start driving, so that the control room knows I am responding.

ER-05 : As an emergency responder, I want to mark an incident as "Resolved" after rescue operations are complete, so that the system can close the incident.

ER-06 : As an emergency responder, I want to receive updates if the incident severity changes (e.g., fire spreads), so that I can request additional resources.

ER-07 : As a firefighter, I want to see nearby hydrant locations on the map, so that I can plan water supply.

3.3 User Stories – System Administrator

AD-01: As an admin, I want to register new cameras with name, location address, GPS coordinates, and stream URL, so that they can be monitored by the system.

AD-02 : As an admin, I want to adjust detection sensitivity per camera (low/medium/high), so that false alarms are minimized in low-risk zones.

AD-03 : As an admin, I want to view system logs and audit trails (who did what, when), so that I can track suspicious activity or troubleshoot issues.

AD-04 : As an admin, I want to generate incident reports filtered by date range, camera zone, incident type, and severity, so that performance can be analyzed.

AD-05 :As an admin, I want to manage user roles and permissions (Admin, Operator, Responder, Viewer), so that access control is enforced.

AD-06 : As an admin, I want to receive alerts when a camera goes offline or the detection service stops, so that I can fix problems quickly.

AD-07 : As an admin, I want to configure which emergency services receive notifications (e.g., only fire for fire incidents, only police for accidents), so that alerts are relevant.

AD-08 : As an admin, I want to set up privacy masks (draw rectangles on video feed that are blurred), so that private properties are not recorded.

AD-09 : As an admin, I want to export incident data in CSV format for external analysis, so that municipal authorities can create their own reports.

AD-10 : As an admin, I want to test detection by uploading sample videos, so that I can validate system performance without live incidents.

3.4 User Stories – Municipal Authority

MA-01 : As a municipal authority, I want to view a city-wide incident heatmap showing where fires and accidents occur most frequently, so that I can identify high-risk areas.

MA-02 : As a municipal authority, I want to see average emergency response time before and after system deployment, so that I can measure effectiveness.

MA-03: As a municipal authority, I want to export incident data for budget planning, so that I can justify additional safety investments.

MA-04 : As a municipal authority, I want to receive monthly summary reports (PDF) via email, so that I can stay informed without logging into the system.

MA-05: As a municipal authority, I want to compare incident statistics across different zones of the city, so that I can allocate resources equitably.

3.5 User Stories – General Public

GP-01 : As a citizen, I want to be assured that my privacy is protected (no recording inside homes), so that I support the system.

GP-02 : As a citizen, I want to know that emergency services will be dispatched quickly if an accident happens near me, so that I feel safer.

GP-03 : As a citizen, I want to access public incident statistics (anonymized), so that I can make informed decisions about commuting routes.

4. Use Case Scenarios

Use case scenarios provide a high-level view of system functionality across different modules. The following table maps system levels to modules and use cases.

Level-0 (System)	Level-1 (Module)	Level-2 (Use Case)	Primary Actors
CCTV Emergency Detection System	Camera Management	Register Camera	Admin

Level-0 (System)	Level-1 (Module)	Level-2 (Use Case)	Primary Actors
		Update Camera Info	Admin
		Delete Camera	Admin
		View Camera Status	Operator, Admin
		Configure Privacy Mask	Admin
		Set Detection Sensitivity	Admin
	Detection Engine	Detect Fire	System (automated)
		Detect Accident	System (automated)
		Classify Fire Severity	System
		Classify Accident Severity	System
		Capture Video Clip	System
	Alert Management	Generate Alert	System
		Display Alert to Operator	System
		Verify Incident	Operator
		Mark False Alarm	Operator
		Send Notification to Emergency Services	System, Operator

Level-0 (System)	Level-1 (Module)	Level-2 (Use Case)	Primary Actors
Incident Management	Incident Management	Escalate Unverified Alert	System
		View Incident Details	Operator, Responder, Admin
		View Live Feed	Operator, Responder
		Search Incidents	Operator, Admin
		Export Incident Report	Admin
		Add Manual Incident	Operator
		Mark Incident Resolved	Responder
		Create User	Admin
		Assign Role	Admin
		Update Profile	All
User Management	User Management	Authenticate Login	All
		Reset Password	All
		View Live Feeds (grid)	Operator
		View Alert Queue	Operator
Dashboard	Dashboard	View Heatmap	Operator, Admin, Authority

Level-0 (System)	Level-1 (Module)	Level-2 (Use Case)	Primary Actors
		View Response Time Analytics	Admin, Authority
		View System Health	Admin
	System Configuration	Set Global Parameters	Admin
		Configure Alert Recipients	Admin
		Manage Privacy Zones	Admin
		Manage Backup Settings	Admin

5. Use Case Descriptions

Each use case is described in detail following a standard template: Primary Actor, Goal in Context, Preconditions, Trigger, Main Scenario, Exceptions, Priority, and When Available.

5.1 Camera Management Use Cases

5.1.1 Use Case: Register Camera

Use Case ID: UC-CAM-01

Primary Actor: System Administrator

Goal in Context: To add a new CCTV camera to the system with all necessary configuration.

Preconditions:

- Admin is logged in with Administrator role.
- Camera is physically installed and powered on.
- Camera stream URL (RTSP/HTTP) is known and accessible from the server.

Trigger: Admin needs to onboard a new camera after installation.

Main Scenario:

1. Admin navigates to Camera Management → Register New Camera.
2. System displays a registration form.
3. Admin enters: camera name, location description, GPS latitude, GPS longitude, stream URL.
4. Admin selects default detection sensitivity (Low/Medium/High).
5. Admin optionally draws privacy mask rectangles on a preview image.
6. Admin clicks "Test Stream" to verify connectivity.
7. System attempts to connect to the stream URL.
8. System confirms successful connection and shows a live preview.
9. Admin clicks "Register".
10. System saves camera details and assigns a unique camera ID.
11. Camera appears in the operator dashboard within 10 seconds.

Exceptions:

- Invalid stream URL: System shows "Cannot connect to stream. Check URL and network."
- Duplicate camera name: System suggests adding a suffix (e.g., "North Gate 2").
- GPS coordinates out of city bounds: Warning but still allowed.

Priority: Essential

When Available: First increment

5.1.2 Use Case: Configure Detection Sensitivity

Use Case ID: UC-CAM-02

Primary Actor: System Administrator

Goal in Context: To adjust how sensitive the AI is to potential incidents for a specific camera.

Preconditions: Camera is registered.

Trigger: Too many false alarms from a camera, or missed detections.

Main Scenario:

1. Admin selects a camera from the list.
2. Clicks "Detection Settings".
3. System shows current sensitivity level and a slider (0-100).
4. Admin adjusts slider. A preview shows estimated false positive vs. true positive trade-off.
5. Admin clicks "Apply".
6. System updates the detection pipeline for that camera.
7. System logs the change.

Exceptions: None.

Priority:Expected

When Available:First increment

5.2 Detection Engine Use Cases

5.2.1 Use Case: Detect Fire

Use Case ID: UC-DET-01

Primary Actor: System (automated)

Goal in Context: To identify fire incidents from live video feeds in real-time with minimal latency.

Preconditions:

- Camera is active and streaming.
- Detection service is running.
- AI model is loaded in memory.

Trigger: Video frame contains flame-like colors and movement patterns, or smoke-like texture.

Main Scenario:

1. System captures a video frame from the camera stream (every 100ms).
2. Frame is resized and normalized (e.g., 640x640 pixels).
3. AI model (trained on fire/smoke datasets) performs inference.
4. If confidence score > threshold (default 0.75), potential fire is flagged.
5. System analyzes 5 consecutive frames to reduce false positives.
6. If fire confirmed across multiple frames:
 - a. Incident record is created with type = "Fire".
 - b. Severity classifier runs (small/medium/large/critical based on flame area percentage).
 - c. System captures video clip: 10 seconds before detection + 30 seconds after.
7. System triggers alert generation (see UC-ALT-01).

Exceptions:

- No fire detected: Continue monitoring.
- Model inference time > 200ms: Log performance warning.
- Camera stream interrupted: Mark camera offline, no detection.

Priority: Essential

When Available: First increment

5.2.2 Use Case: Detect Accident

Use Case ID: UC-DET-02

Primary Actor: System (automated)

Goal in Context: To identify road accidents (collisions, rollovers, bike accidents, pedestrian impacts) from video feeds.

Preconditions:

- Camera is active and streaming.
- Detection service is running.

Trigger: Sudden change in vehicle trajectory, rapid deceleration, collision impact, or overturned vehicle.

Main Scenario:

1. System captures video frame sequence (10 frames per second).
2. Object detection identifies vehicles, motorcycles, bicycles, pedestrians.
3. Optical flow analysis detects sudden motion changes.
4. If a collision pattern is detected (two bounding boxes overlap with high relative velocity):
 - a. Incident record is created with type = "Accident".
 - b. Severity classifier runs (minor/major/critical based on vehicle speed, number of vehicles, presence of fire).
5. If rollover detected (vehicle bounding box orientation > 45 degrees from upright):
 - a. Severity automatically set to at least "Major".
6. System captures video clip (10 seconds before + 30 seconds after).

7. System triggers alert generation.

Exceptions:

- Low light conditions: System uses alternative model trained on night data.
- Heavy rain/snow: System logs "poor visibility" and may lower confidence.

Priority: Essential

When Available: First increment

5.3 Alert Management Use Cases

5.3.1 Use Case: Generate Alert

Use Case ID: UC-ALT-01

Primary Actor: System

Goal in Context: To notify human operators and emergency services about a detected incident as quickly as possible.

Preconditions: Fire or accident has been detected and incident record created.

Trigger: Incident status changed to "Detected".

Main Scenario:

1. System creates an Alert object with fields:
 - Alert ID (unique)
 - Incident ID (reference)
 - Timestamp (detection time)
 - Status: "Pending Verification"
2. System pushes alert to operator dashboard via WebSocket (latency < 1 second).
3. Operator dashboard:

- Plays a distinctive audio alert (different for fire vs. accident).
- Shows a pop-up notification with thumbnail.
- Adds alert to the "Pending" queue.

4. System logs the alert generation.

5. System starts a 30-second timer. If not verified within 30 seconds, alert escalates (see UC-ALT-04).

Exceptions:

- Operator dashboard disconnected: System retries every 5 seconds.
- Multiple alerts from same camera within 10 seconds: Deduplicate.

Priority: Essential

When Available: First increment

5.3.2 Use Case: Verify Incident

Use Case ID: UC-ALT-02

Primary Actor: CCTV Operator

Goal in Context: To confirm whether a detected incident is a genuine emergency or a false alarm.

Preconditions: Alert status is "Pending Verification".

Trigger: Operator clicks on the alert notification.

Main Scenario:

1. Operator opens the alert detail view.
2. System displays:
 - Live feed from the camera (if still active)
 - Recorded clip from the incident
 - Incident type, severity, location, timestamp

3. Operator visually inspects the footage.

4. Operator clicks one of:

- Confirm (incident is real)
- False Alarm (incident is not real)

5. If Confirm:

- System prompts optional notes (e.g., "Fire spreading to second floor").
- Alert status changes to "Confirmed".
- System triggers emergency dispatch (UC-ALT-03).

6. If False Alarm:

- System prompts for a reason (drop-down: "Steam/Exhaust", "Shadow", "Animal", "Other").
- Alert status changes to "False Alarm".
- No dispatch occurs.
- System logs the false alarm for model retraining.

Exceptions:

- Operator takes more than 30 seconds: System escalates to supervisor.
- Video feed unavailable: Operator relies on recorded clip only.

Priority: Essential

When Available: First increment

5.3.3 Use Case: Send Notification to Emergency Services

Use Case ID: UC-ALT-03

Primary Actor: System (triggered by operator confirmation)

Goal in Context: To automatically inform relevant emergency services with all necessary data.

Preconditions: Alert status is "Confirmed".

Trigger: Operator clicks Confirm.

Main Scenario:

1. System determines which services to notify based on incident type:
 - Fire → Fire brigade
 - Accident → Ambulance + Police
2. For each service, system composes a notification payload:
 - Incident ID, type, severity
 - GPS coordinates (latitude, longitude)
 - Address (reverse geocoded)
 - Live feed URL (expiring token)
 - Video clip download URL
 - Estimated time of detection
3. System sends notification via:
 - Push notification to responder mobile apps (if configured)
 - HTTP API to existing Computer-Aided Dispatch (CAD) system
 - Email/SMS as fallback
4. System logs each notification with timestamp and delivery status.
5. Alert status updates to "Dispatched".

Exceptions:

- API failure: Retry 3 times with exponential backoff. If still fails, send SMS.
- No configured recipients: System alerts admin.

Priority: Essential

When Available: Second increment (API integration may be delayed)

5.4 Incident Management Use Cases

5.4.1 Use Case: View Incident Details

Use Case ID:UC-INC-01

Primary Actor: Operator, Responder, Admin

Goal in Context: To access complete information about a past or ongoing incident.

Preconditions: Incident exists in the database.

Trigger: User selects an incident from list or search results.

Main Scenario:

1. User clicks on an incident (from dashboard, search, or alert history).
2. System displays incident detail page with sections:
 - Summary: Type, severity, timestamp, camera name, location
 - Video: Embedded player with recorded clip (10 sec before + 30 sec after)
 - Live feed button (if incident is still active and camera online)
 - Timeline: Status changes (Detected → Confirmed → Dispatched → Responding → Resolved)
 - Verification details: Operator name, verification timestamp, notes
 - Notification log: Which services were notified, when, and delivery status
3. User can play the video, pause, seek, and download if permitted.
4. User can return to the list.

Priority: Essential

When Available: First increment

5.4.2 Use Case: Add Manual Incident

Use Case ID:UC-INC-02

Primary Actor: CCTV Operator

Goal in Context: To report an incident that the AI missed (e.g., very small fire or accident outside camera view).

Preconditions: Operator is logged in.

Trigger: Operator sees an incident while monitoring live feed that did not trigger an alert.

Main Scenario:

1. Operator clicks "Report Incident" button on the dashboard.
2. System displays a form.
3. Operator selects camera (or enters location if not camera-based).
4. Operator selects incident type (Fire / Accident).
5. Operator selects severity.
6. Operator can upload a screenshot or short video (optional).
7. Operator enters description.
8. Operator clicks "Submit".
9. System creates an incident record with source = "Manual".
10. System generates alert and proceeds with verification (operator can self-verify).
11. Incident is visible to admins and responders.

Priority: Expected

When Available: Second increment

5.4.3 Use Case: Export Incident Report

Use Case ID: UC-INC-03

Primary Actor: Admin, Municipal Authority

Goal in Context: To generate a formal report for analysis, legal evidence, or budget planning.

Preconditions: User has Export permission.

Trigger: User needs a report for a specific date range.

Main Scenario:

1. User navigates to Reports → Export.
2. System displays filter options:
 - Date range (start to end)
 - Incident type (Fire, Accident, or both)
 - Severity (any or specific)
 - Camera zone (optional)
 - Verification status (Confirmed, False Alarm, Pending)
3. User selects filters and output format (PDF or CSV).
4. User clicks "Generate".
5. System queries the database and compiles results.
6. For PDF: System generates formatted report with summary tables, charts, and incident list.
7. For CSV: System generates raw data for external analysis.
8. User downloads the file.

Exceptions: No incidents in selected range → report shows "No data".

Priority: Expected

When Available: Second increment

[5.5 Dashboard & Analytics Use Cases](#)

5.5.1 Use Case: View Incident Heatmap

Use Case ID: UC-DASH-01

Primary Actor: Operator, Admin, Municipal Authority

Goal in Context: To visually identify high-risk areas (clusters of incidents) on a map.

Preconditions: At least one incident has been recorded.

Trigger: User opens the Heatmap tab.

Main Scenario:

1. System displays a map (OpenStreetMap or Google Maps) of the city.
2. Overlay shows color-coded heatmap:
 - Red: High density of incidents
 - Yellow: Medium density
 - Green: Low density
3. User can filter by:
 - Incident type (Fire only, Accident only, or both)
 - Date range (last 7 days, 30 days, custom)
 - Severity (Minor, Major, Critical)
4. As filters change, heatmap updates dynamically.
5. User can click on a hotspot to see a list of incidents in that area.

Priority: Exciting

When Available: Third increment

5.5.2 Use Case: View Response Time Analytics

Use Case ID: UC-DASH-02

Primary Actor: Admin, Municipal Authority

Goal in Context: To measure system effectiveness in reducing emergency response time.

Preconditions: At least 30 days of data.

Trigger: User opens Analytics dashboard.

Main Scenario:

1. System displays key performance indicators (KPIs):
 - Average detection-to-alert time (target: < 5 seconds)
 - Average operator verification time (target: < 15 seconds)
 - Average emergency dispatch-to-arrival time (target: depends on zone)
 - False positive rate (target: < 10%)
2. Charts show trends over time (daily, weekly, monthly).
3. User can compare pre-deployment vs. post-deployment response times (if historical data available).
4. System optionally shows a "lives saved estimate" based on reduced response times (using statistical models).

Priority: Expected

When Available: Third increment

5.6 User & Role Management Use Cases

5.6.1 Use Case: Create User

Use Case ID: UC-USER-01

Primary Actor: Admin

Goal in Context: To add a new user to the system with appropriate role.

Preconditions: Admin is logged in.

Trigger: New personnel (operator, responder, or other admin) joins.

Main Scenario:

1. Admin navigates to User Management → Add User.
2. System displays form.
3. Admin enters:
 - Full name
 - Email address (used for login and notifications)
 - Phone number (for SMS alerts)
 - Role (Admin, Operator, Responder, Viewer)
 - Camera access restrictions (optional: limit to specific cameras/zones)
4. Admin clicks "Create".
5. System generates a temporary password.
6. System sends an email to the user with login instructions and temporary password.
7. User is prompted to change password on first login.

Priority: Essential

When Available: First increment

6. Use Case Diagrams

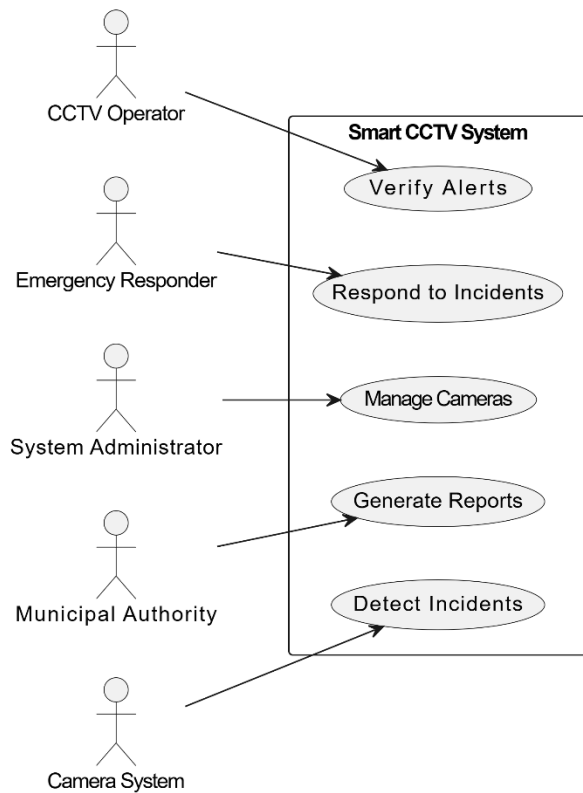
Level 0: System Context Diagram

Actors: CCTV Operator, Emergency Responder, System Administrator, Municipal Authority, Camera System (external)

Use Cases (high-level):

- Manage Cameras

- Detect Incidents
- Verify Alerts
- Respond to Incidents
- Generate Reports



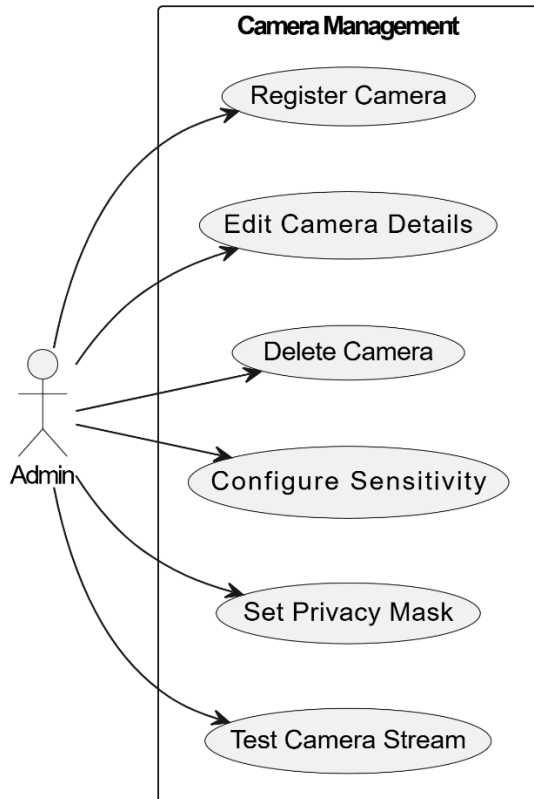
Level 2.1: Camera Management (Admin)

Actor: Admin

Use Cases:

- Register Camera
- Edit Camera Details

- Delete Camera
- Configure Sensitivity
- Set Privacy Mask
- Test Camera Stream



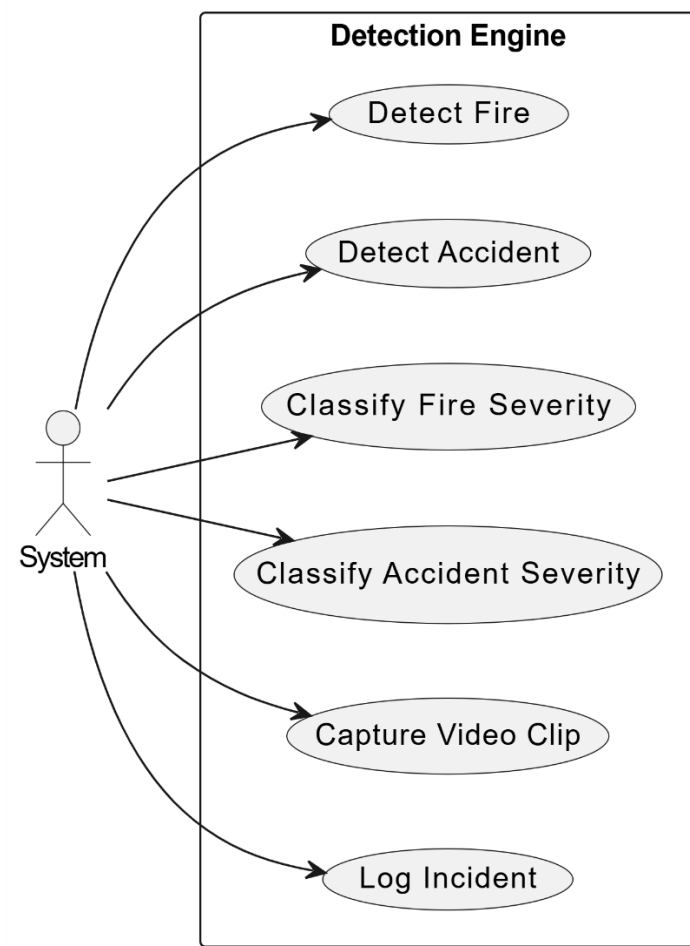
Level 2.2: Detection Engine (System)

Actor: System (automated)

Use Cases:

- Detect Fire
- Detect Accident
- Classify Fire Severity

- Classify Accident Severity
- Capture Video Clip
- Log Incident



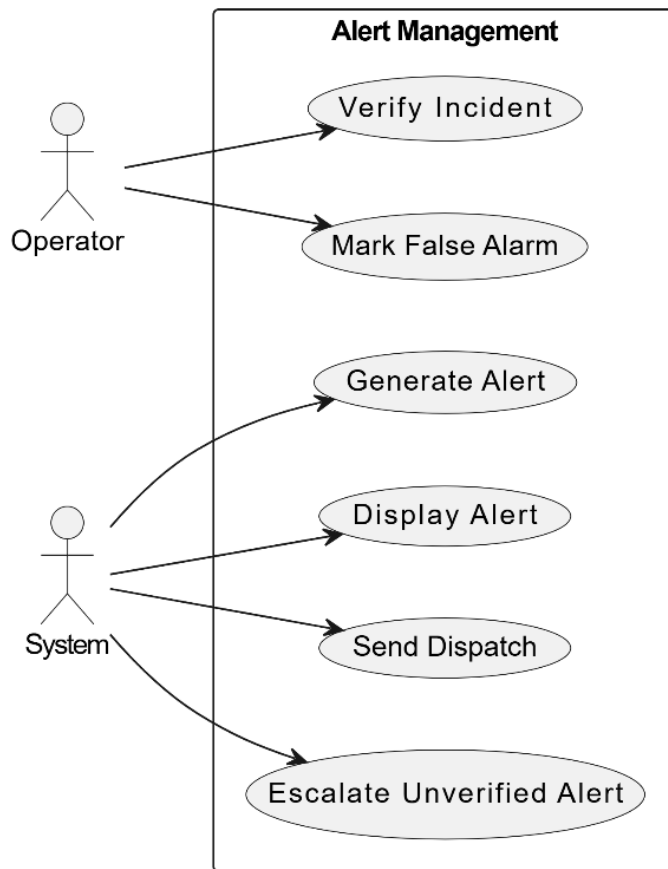
Level 2.3: Alert Management

Actors: Operator, System

Use Cases:

- Generate Alert

- Display Alert
- Verify Incident (Operator)
- Mark False Alarm (Operator)
- Send Dispatch (System)
- Escalate Unverified Alert (System)

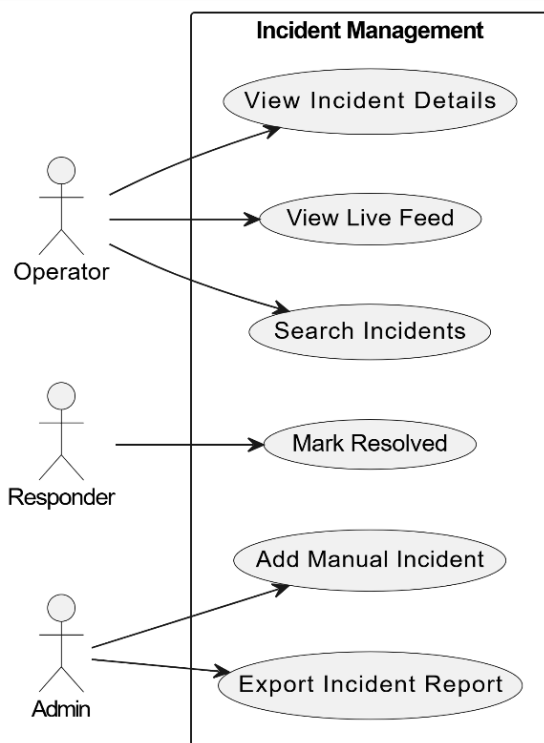


Level 2.4: Incident Management

Actors: Operator, Responder,

Use Cases:

- View Incident Details
- View Live Feed
- Search Incidents
- Export Incident Report
- Add Manual Incident
- Mark Resolved



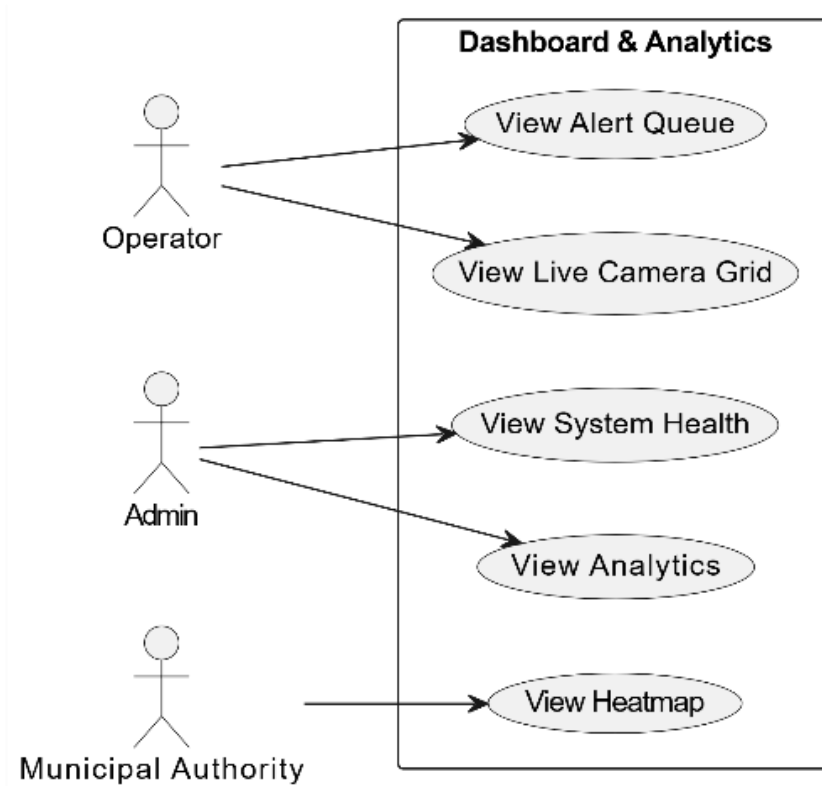
Level 2.5: Dashboard & Analytics

Actors: Operator, Admin, Municipal Authority

Use Cases:

- View Live Camera Grid

- View Alert Queue
- View Heatmap
- View Analytics
- View System Health



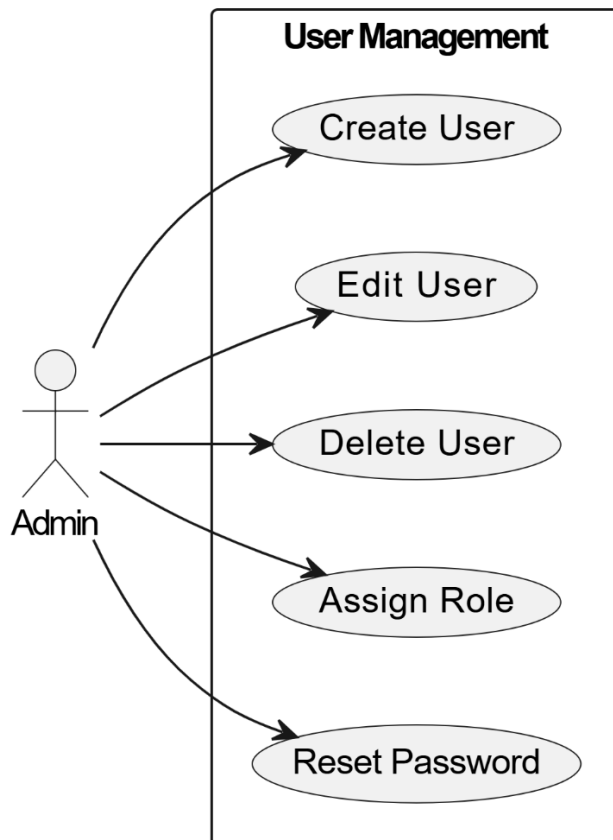
Level 2.6: User Management

Actor: Admin

Use Cases:

- Create User
- Edit User
- Delete User

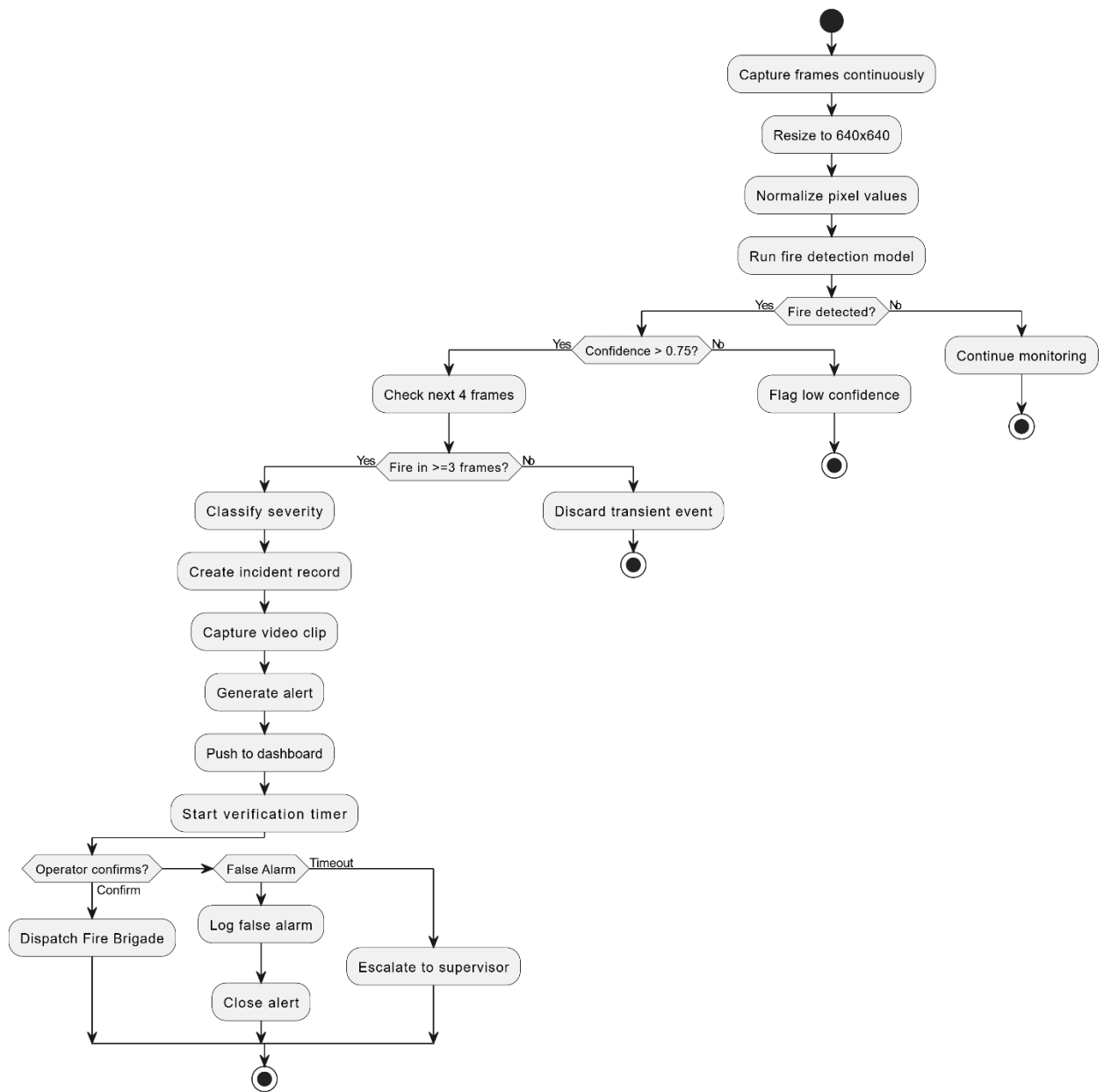
- Assign Role
- Reset Password



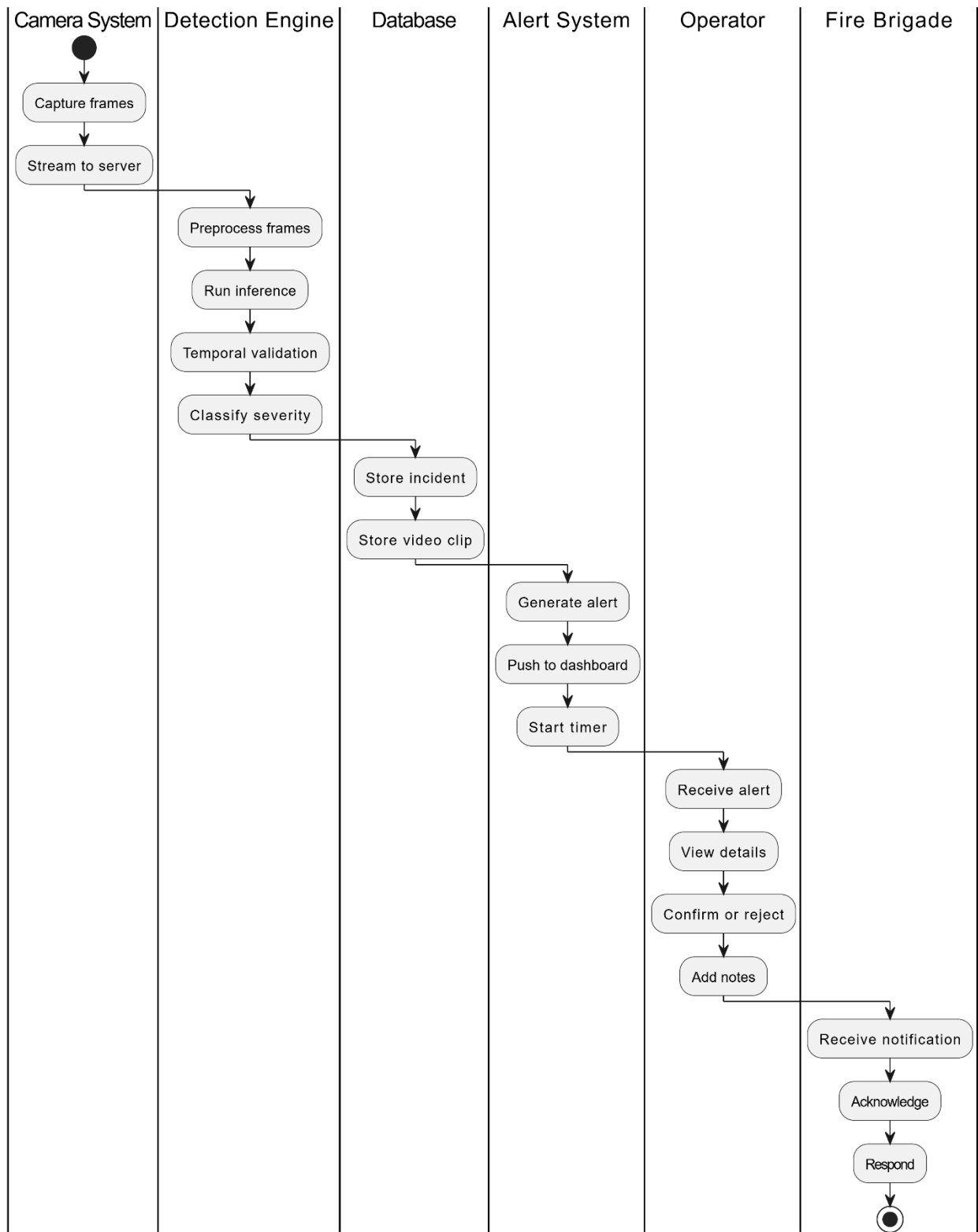
7. Activity & Swimlane Diagrams

7.1 Fire Detection and Alert Flow

Activity Diagram :

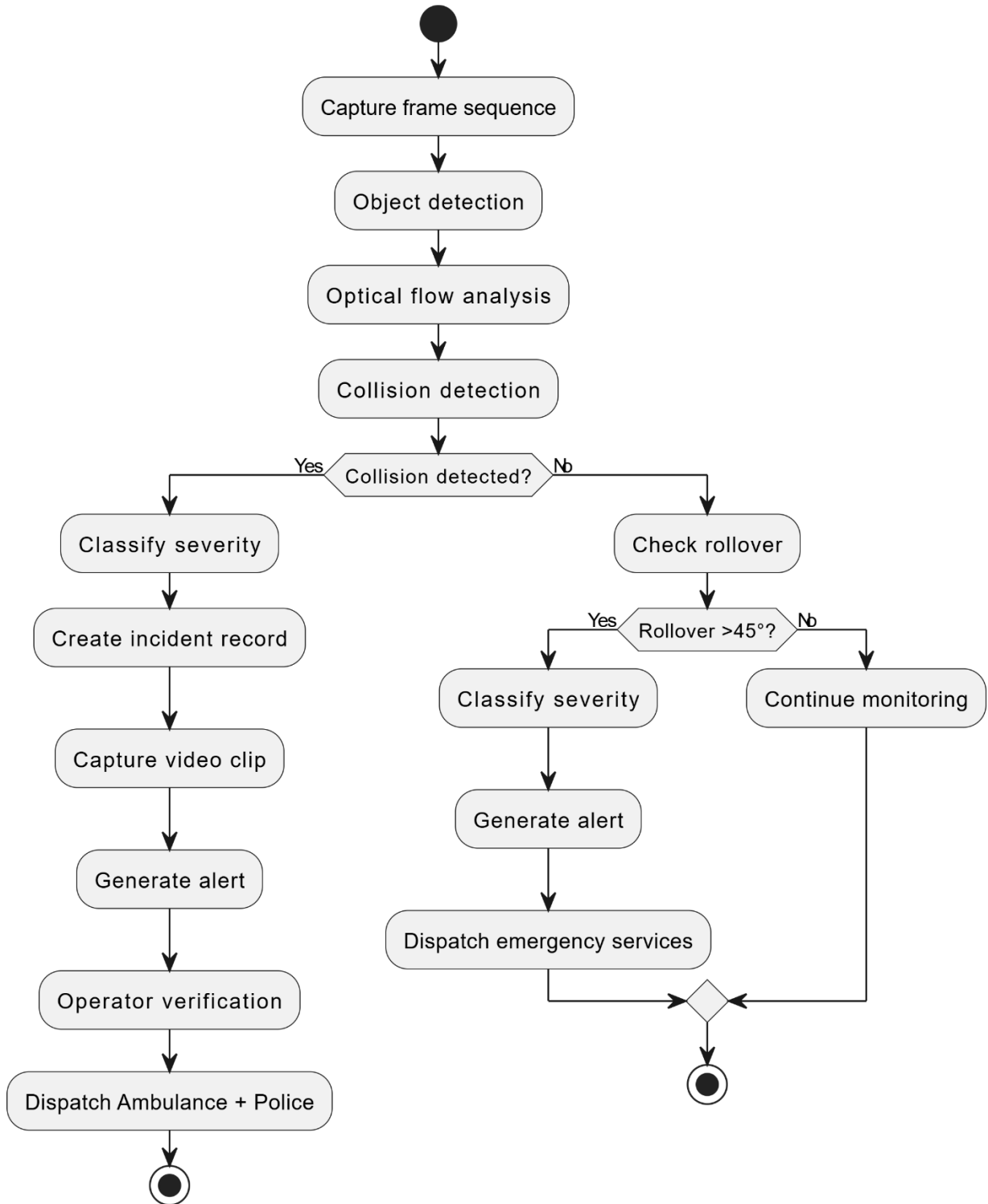


Swimlane Diagram :

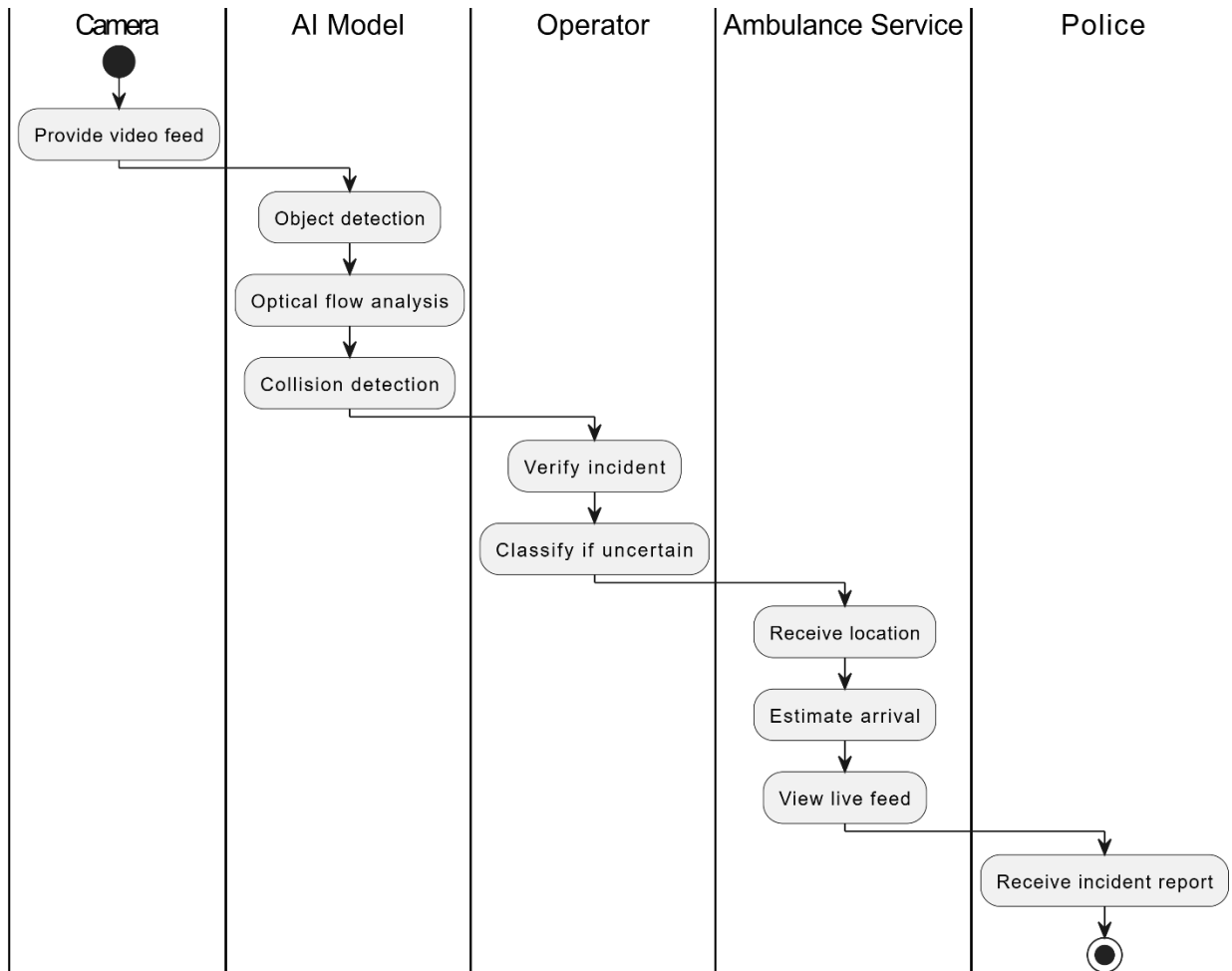


7.2 Accident Detection and Response Flow

Activity Diagram:

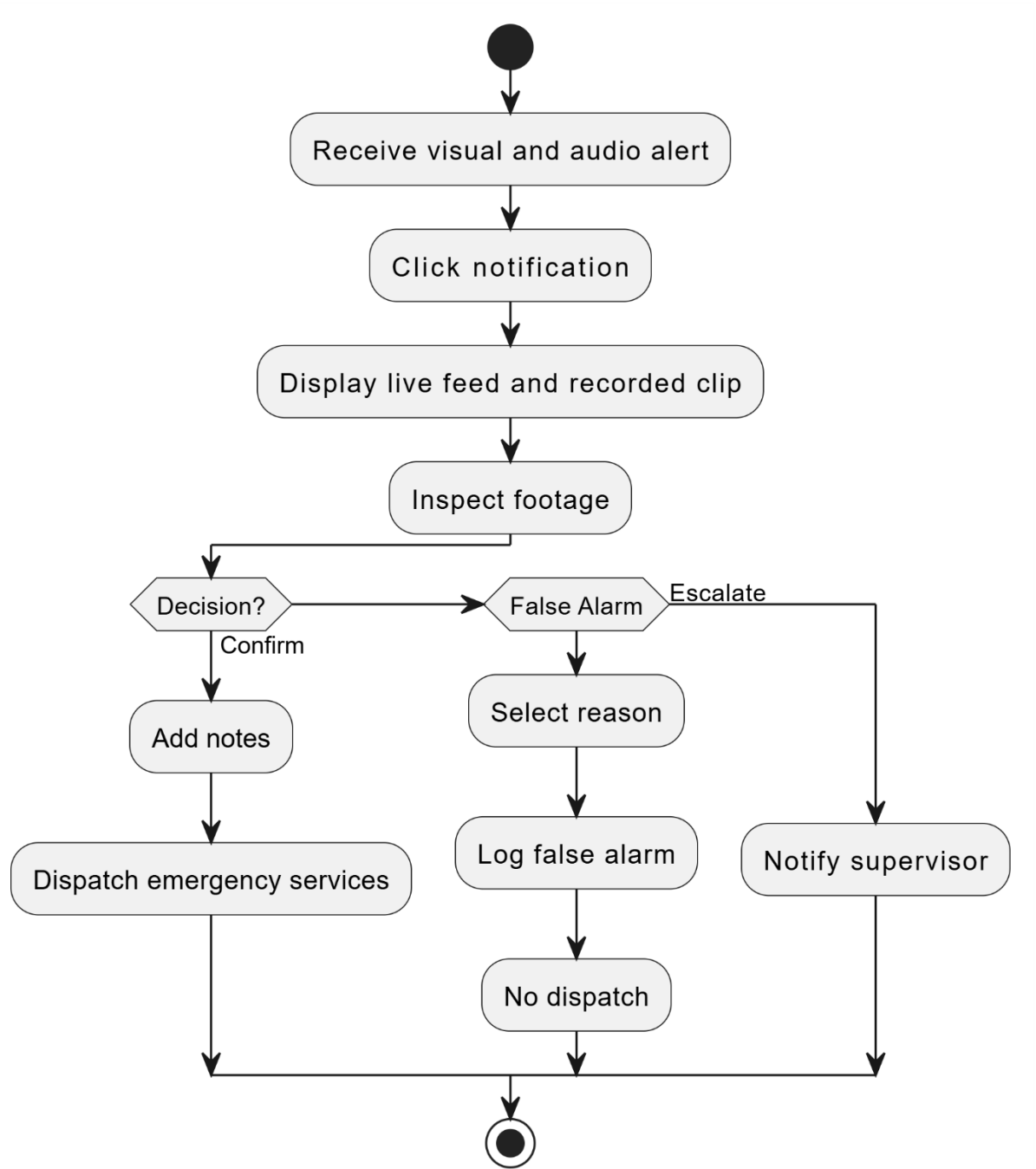


Swimlane Diagram:

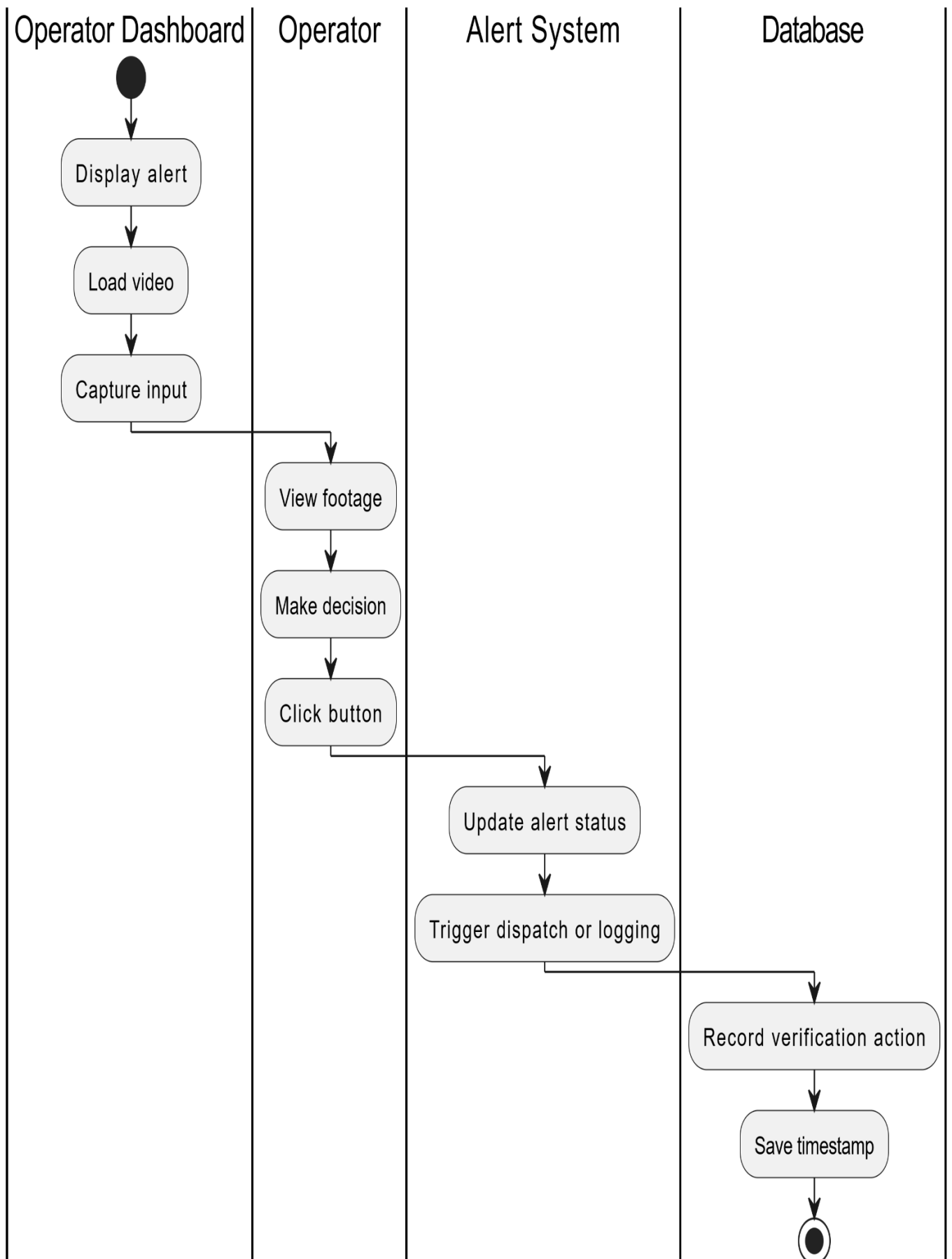


7.3 Operator Verification Flow

Activity Diagram:

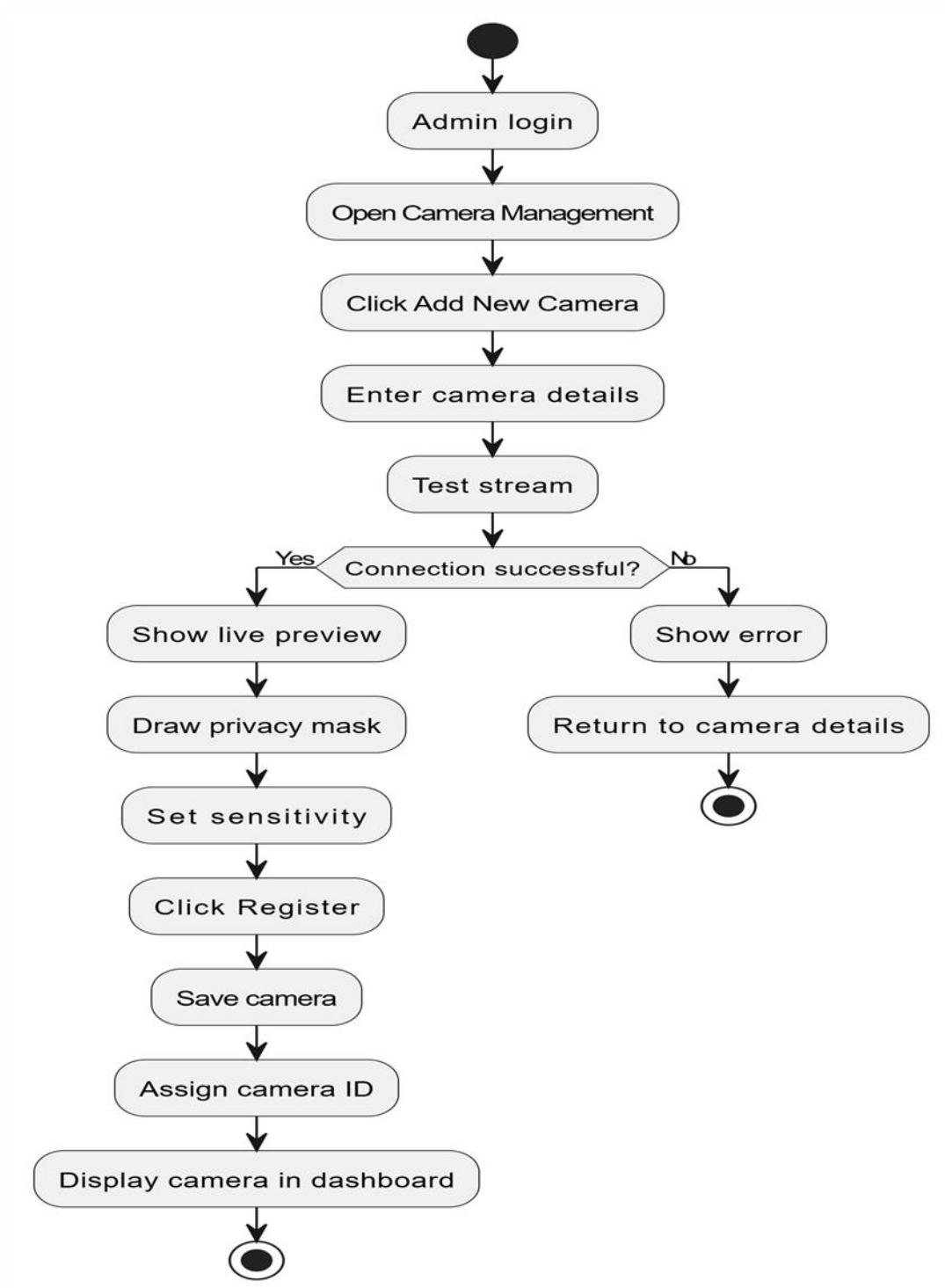


Swimlane Diagram:

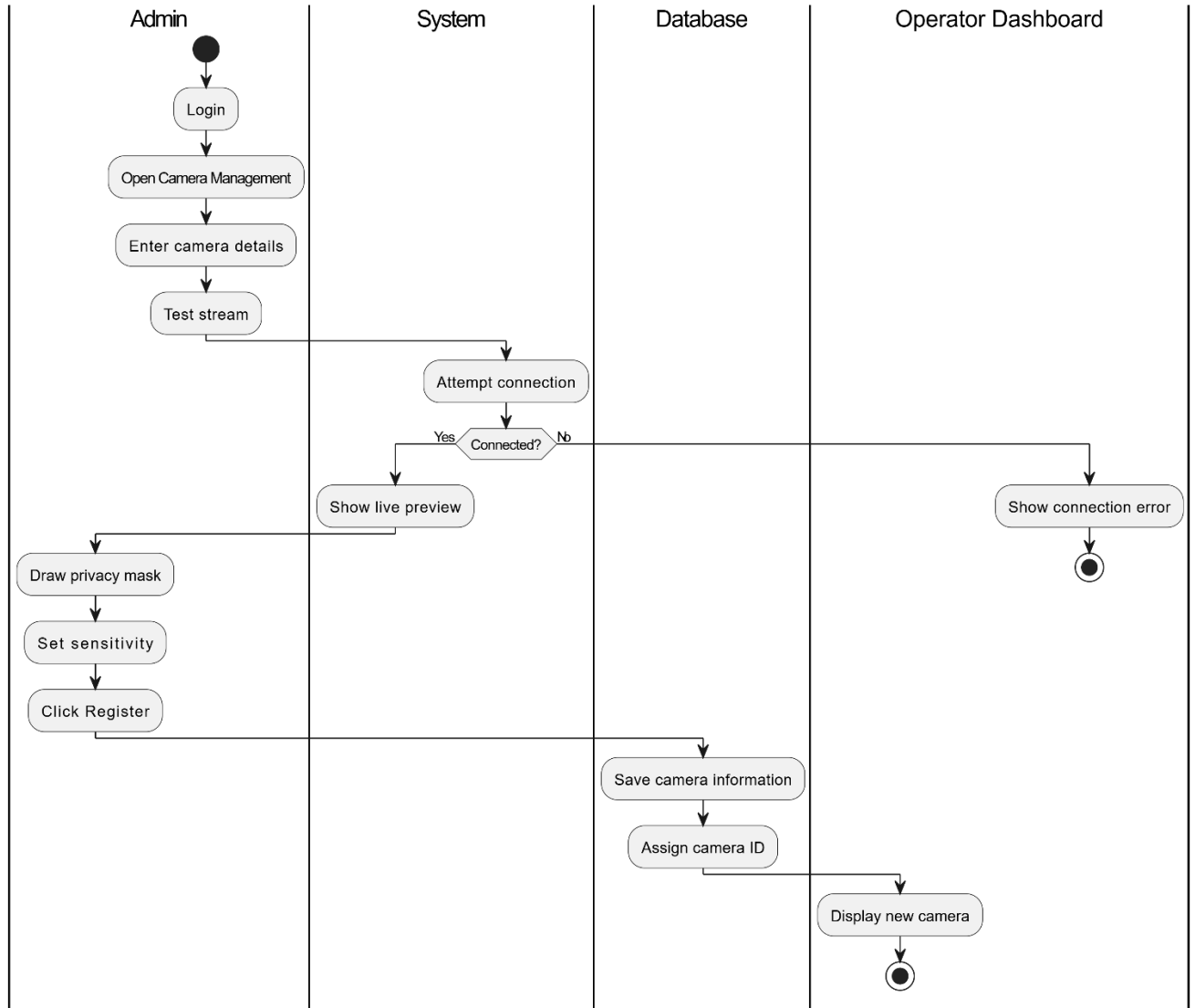


7.4 Admin Camera Management Flow

Activity Diagram:

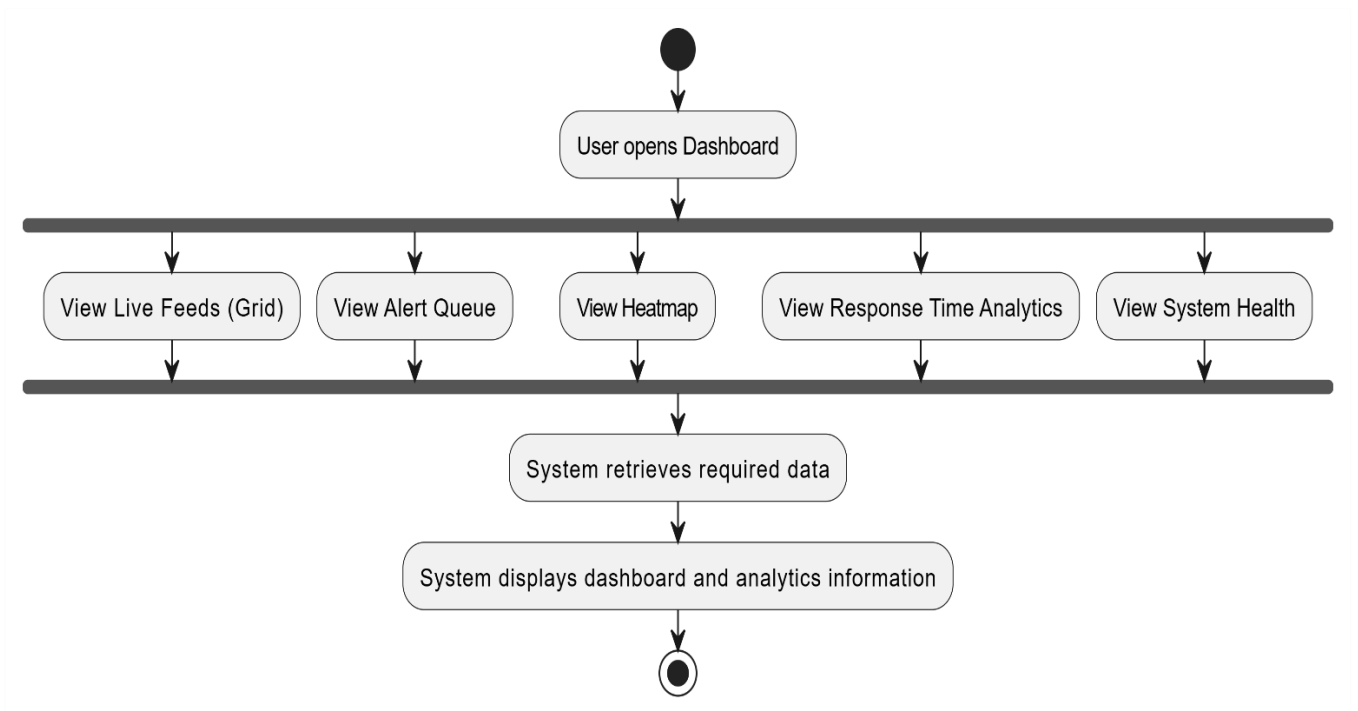


Swimlane Diagram:

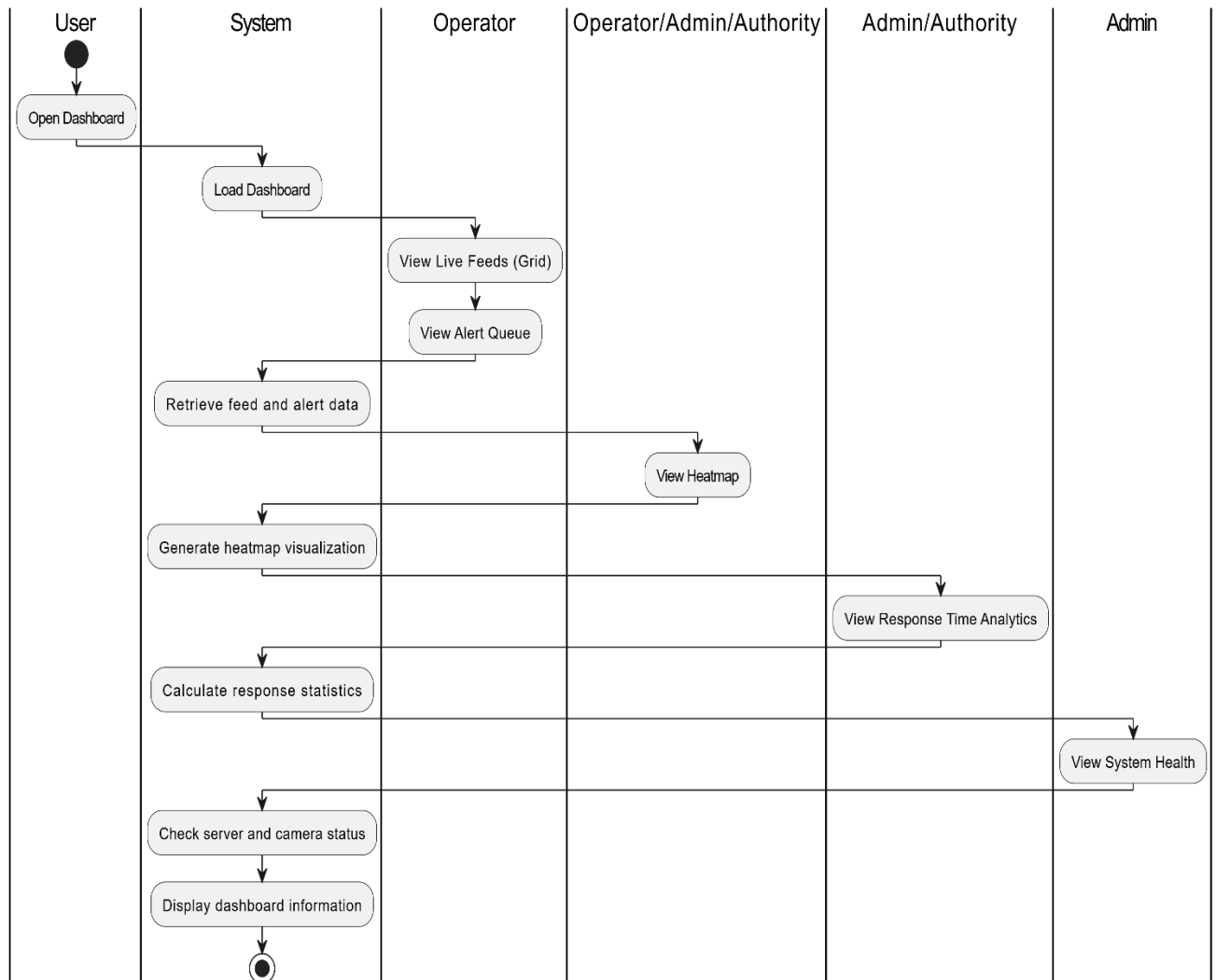


7.4 Dashboard & Analytics

Activity Diagram:



Swimlane Diagram:



8. Entity Relationship (ER) Diagram

8.1 Entity Identification

The following entities are identified from the system model:

Camera : CCTV camera with physical location and stream configuration

User : System user (operator, admin, responder, authority)

Role :User permission set (Admin, Operator, Responder, Viewer)

Incident : Detected fire or accident event

Alert :Notification generated from an incident

Verification : Record of operator's decision on an alert

VideoClip : Recorded footage of an incident

NotificationLog : Log of notifications sent to emergency services

SystemConfig : Global or per-camera configuration settings

CameraGroup : Grouping of cameras by zone (e.g., "Downtown")

8.2 Attributes and Primary Keys

Camera:

camera_id : INT , PRIMARY KEY, AUTO_INCREMENT

camera_name :VARCHAR(100) | NOT NULL, UNIQUE

location_address : TEXT , NOT NULL

User:

user_id : INT , PRIMARY KEY, AUTO_INCREMENT

full_name : VARCHAR(100) , NOT NULL

email : VARCHAR(150) , NOT NULL, UNIQUE

role_id :INT , FOREIGN KEY (Role)

Role:

role_id : INT , PRIMARY KEY

role_name : VARCHAR(50) , NOT NULL, UNIQUE

permissions :TEXT ,NOT NULL

Incident:

incident_id: INT , PRIMARY KEY, AUTO_INCREMENT ,Unique identifier

camera_id : INT , FOREIGN KEY (Camera)

incident_type : ENUM('Fire','Accident') ,NOT NULL

severity : ENUM('Small','Medium','Large','Critical','Minor','Major','Critical') , NOT NULL

detected_at : TIMESTAMP , NOT NULL

source : ENUM('AI','Manual')

video_clip_url : VARCHAR(500) , NULL

thumbnail_url :VARCHAR(500) , NULL

status:ENUM('Detected','Confirmed','FalseAlarm','Dispatched','Responding','Resolved')

location_lat : DECIMAL(10,8) , NOT NULL

location_lng :DECIMAL(11,8) , NOT NULL

address : TEXT , NULL

Alert:

alert_id : INT , PRIMARY KEY, AUTO_INCREMENT Unique identifier

incident_id : INT , FOREIGN KEY (Incident)

generated_at : TIMESTAMP , DEFAULT CURRENT_TIMESTAMP

operator_id : INT ,FOREIGN KEY (User)

verification_status : ENUM('Pending','Confirmed','FalseAlarm') , NOT NULL

verified_at : TIMESTAMP , NULL

verification_notes : TEXT , NULL

false_alarm_reason : VARCHAR(100) , NULL

NotificationLog:

notification_id : INT , PRIMARY KEY, AUTO_INCREMENT ,Unique identifier

incident_id : INT ,FOREIGN KEY (Incident)

service_type : ENUM('Fire','Ambulance','Police') ,NOT NULL

sent_at : TIMESTAMP ,DEFAULT CURRENT_TIMESTAMP

delivery_status : ENUM('Sent','Failed','Pending') , DEFAULT 'Pending'

error_message : TEXT , NULL

SystemConfig:

config_id : INT , PRIMARY KEY

global_sensitivity : ENUM('Low','Medium','High') ,DEFAULT 'Medium'

alert_timeout_seconds : INT , DEFAULT 30

video_clip_pre_seconds : INT, DEFAULT 10

| video_clip_post_seconds : INT , DEFAULT 30

| fire_threshold : FLOAT , DEFAULT 0.75

| accident_threshold : FLOAT , DEFAULT 0.70

8.3 Relationships and Cardinalities

Camera → Incident : One-to-Many (1:M)

Incident → Alert :One-to-One (1:1)

Alert → User (Verification) : Many-to-One (M:1)

Incident → NotificationLog : One-to-Many (1:M)

User → NotificationLog : One-to-Many (1:M)

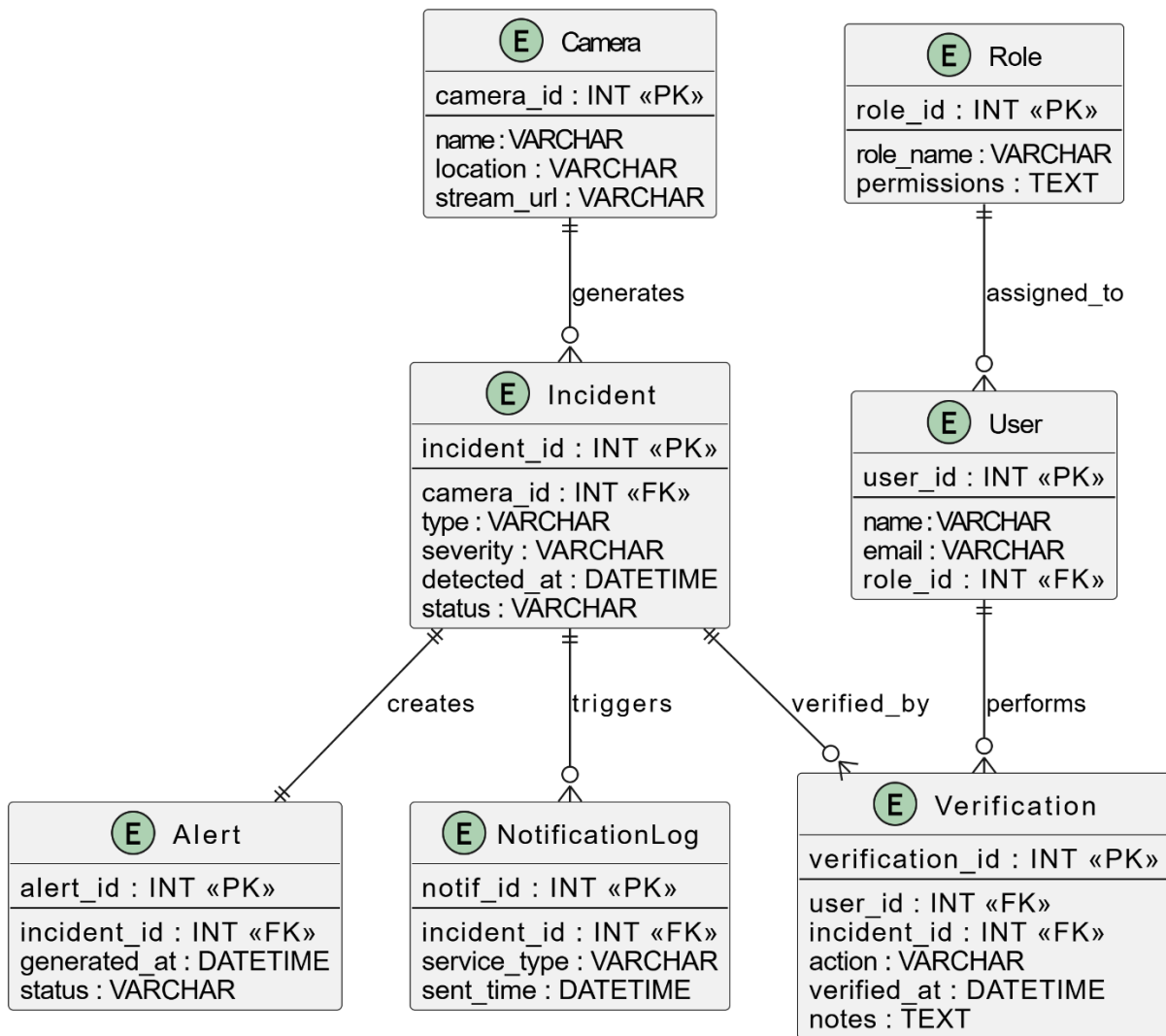
Role → User : One-to-Many (1:M)

Camera → VideoClip :one-to-Many (1:M)

Incident → VideoClip : One-to-One (1:1)

CameraGroup → Camera : One-to-Many (1:M)

8.4 Complete ER Diagram



9. Class-Based Modeling

9.1 Class Identification

Using noun extraction from requirements, we identified candidate classes:

Candidate Class	Accepted?	Reason / Justification
Camera	Yes	Has attributes and operations
User	Yes	Core entity
Incident	Yes	Core event
Alert	Yes	Notification mechanism
DetectionEngine	Yes	Central AI component
NotificationService	Yes	Handles dispatch
Dashboard	Yes	UI component
VideoProcessor	Yes	Handles frame capture
DatabaseConnection	No	Infrastructure, not domain
ConfigManager	Yes	Manages settings
PrivacyMask	Yes	Feature
SeverityClassifier	Yes	Subcomponent
Frame	No	Data structure
StreamURL	No	Attribute

9.2 Detailed Class Attributes and Operations

Class Camera

- cameraId : int , private
- name : string , private
- location : string , private

operation:

- + register() : bool
- + update() : bool
- + delete() : bool

Class Incident

- incidentId :int
- type : enum

Operation :

- + getDetails() : string

Class Alert

- alertId : int
- incidentId : int

Operation :

+ markFalseAlarm() : void

Class NotificationService

- fireAPIEndpoint: string
- ambulanceAPIEndpoint : string
- policeAPIEndpoint : string

Operation :

- + sendToFire() :bool
- + sendToAmbulance() : bool
- + sendToPolice() : bool

Class Dashboard

- liveFeeds : list
- alertQueue : list

Operation :

- + displayLiveFeeds() :void
- + showAlert() : void

9.3 Class Relationships

Camera (1) --- (M) Incident → Composition (camera produces incidents)

Incident (1) --- (1) Alert → One-to-one association

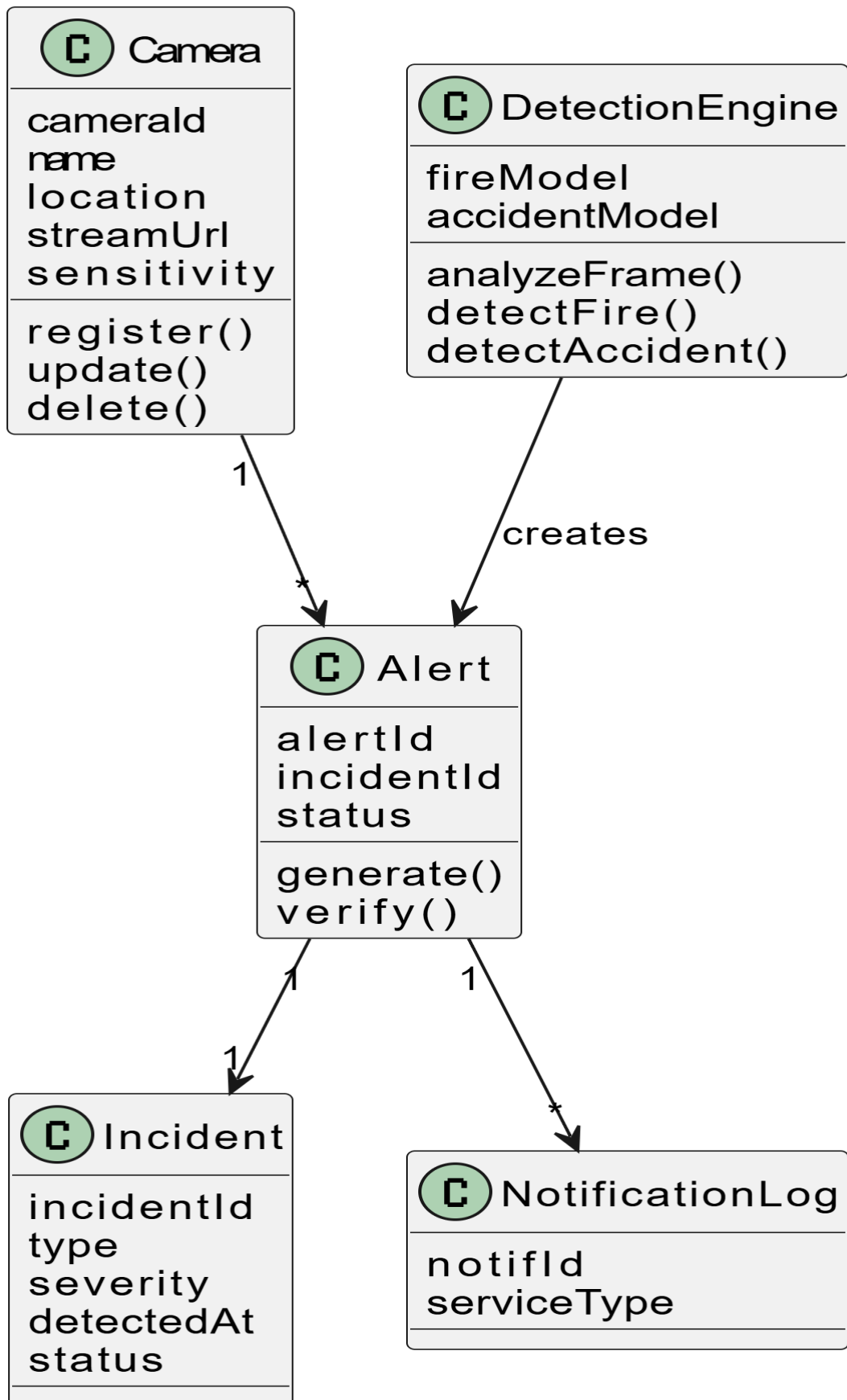
User (1) --- (M) Alert → Association

DetectionEngine (1) --- (M) Incident → Dependency (engine creates incidents)

NotificationService (1) --- (M) NotificationLog → Composition

Dashboard (1) --- (M) Alert → Aggregation (dashboard displays alerts)

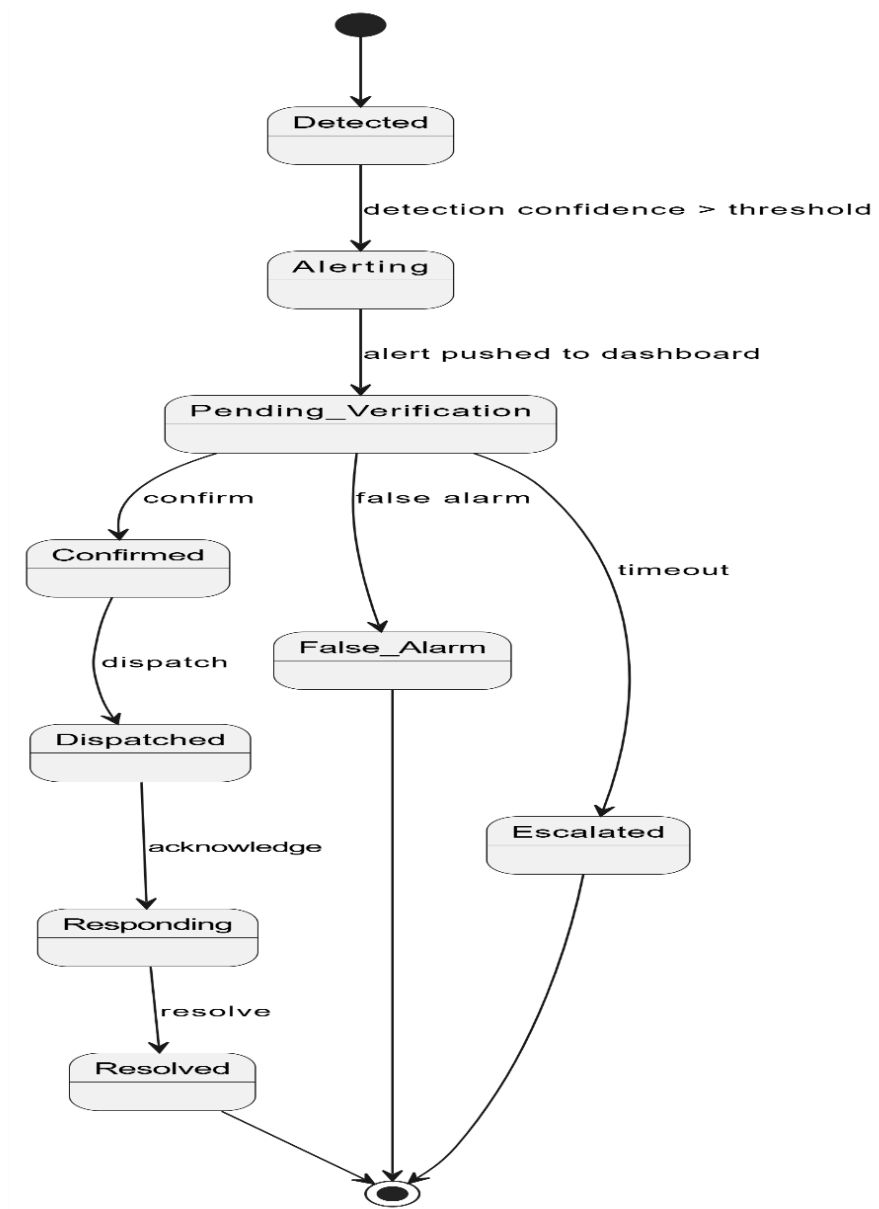
9.4 Class Diagram



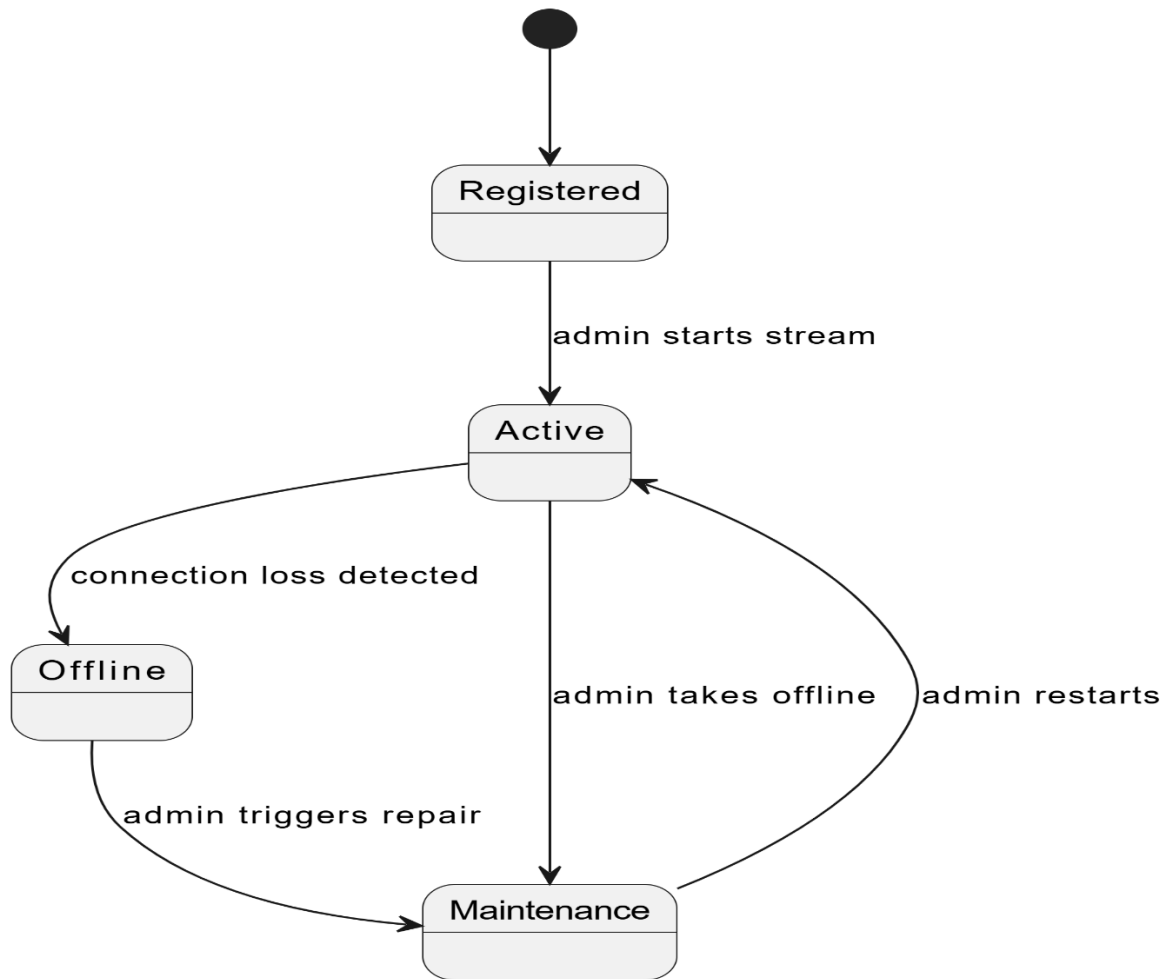
10. Behavioral Modeling

10.1 State Transition Diagrams

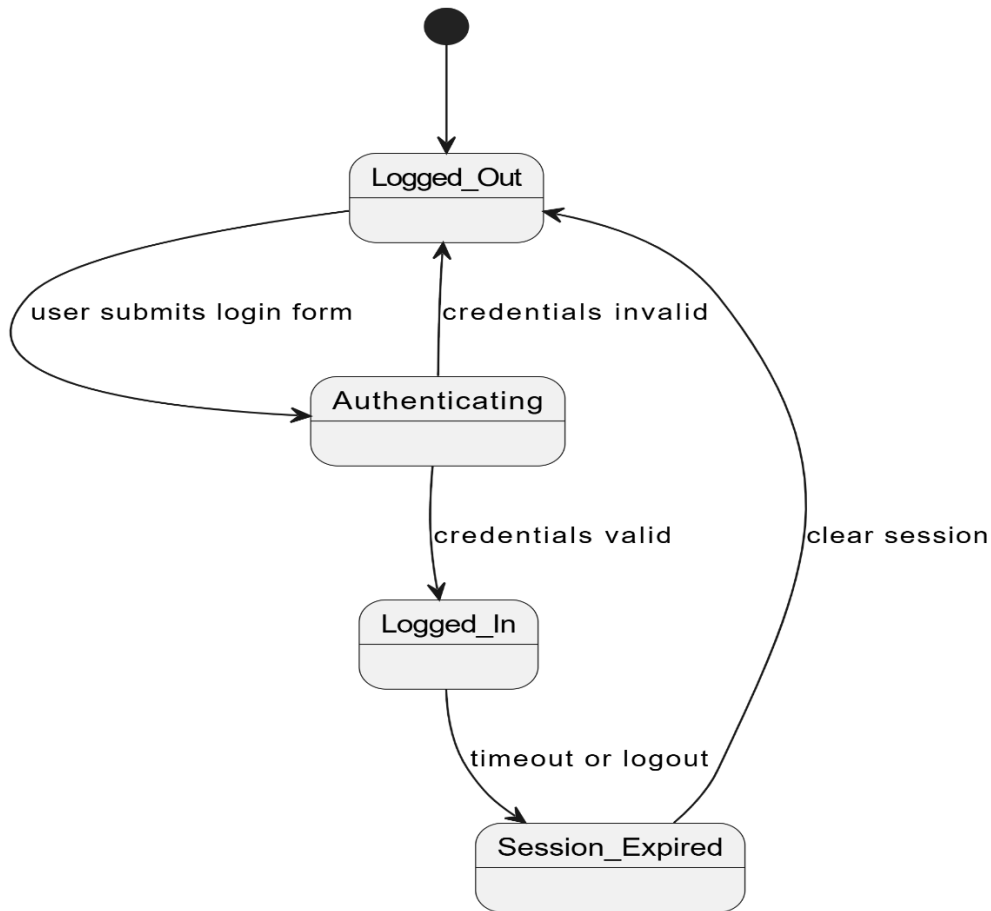
ID: ST-01 – Incident Lifecycle



ID: ST-02 – Camera State



ID: ST-03 – User Session



11. Data Flow Diagrams (DFD)

11.1 Level 0 – Context Diagram

External Entities:

- CCTV Camera – Provides video stream.
- CCTV Operator – Receives alerts, provides verification.
- Emergency Responder – Receives dispatch notifications, provides acknowledgments.
- System Administrator – Configures system, manages cameras.

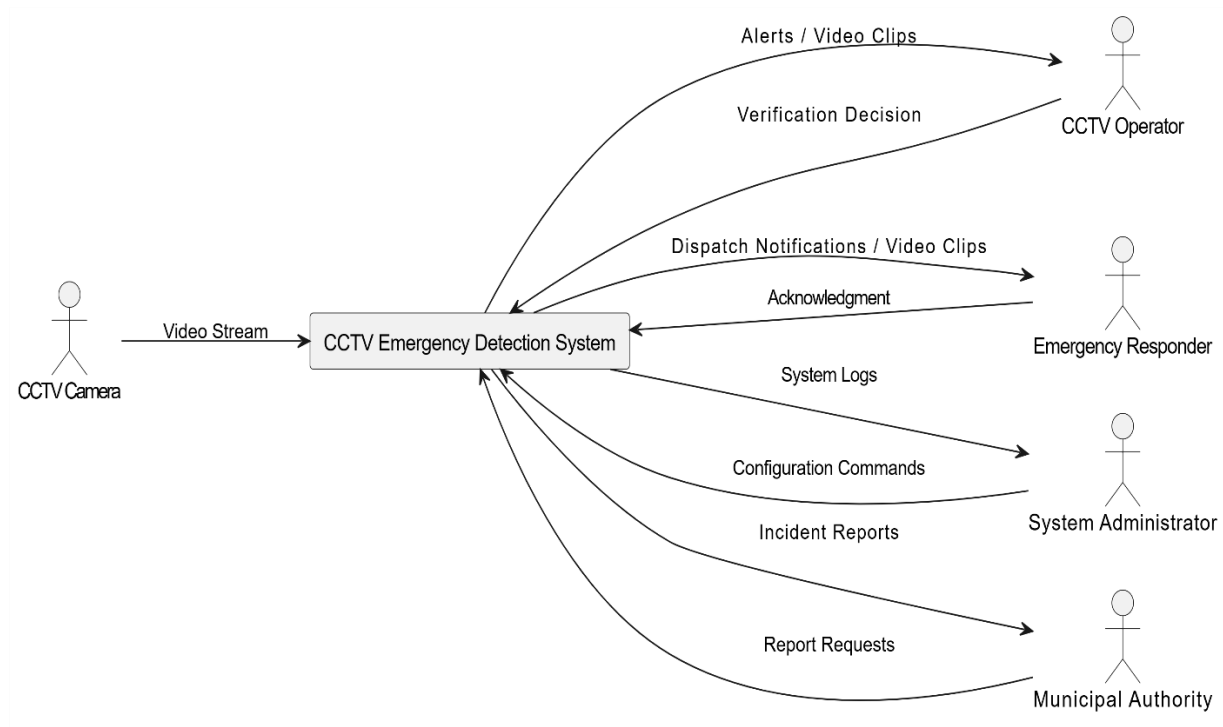
- Municipal Authority – Requests reports.

Data Flows (inputs to system):

- Video stream (from Camera)
- Verification decision (from Operator)
- Acknowledgment (from Responder)
- Configuration commands (from Admin)
- Report requests (from Authority)

Data Flows (outputs from system):

- Alerts (to Operator)
- Dispatch notifications (to Responder)
- Incident reports (to Authority)
- System logs (to Admin)
- Video clip URLs (to Responder/Operator)



11.2 Level 1 – Main Processes

Process 1.0: Video Ingestion

- Input: Video stream (RTSP)
- Output: Individual frames (JPEG) to frame buffer
- Data Store: Camera configuration (D1)

Process 2.0: Detection Analysis

- Input: Video frames
- Output: Detection events (fire/accident)
- Data Store: None (real-time)

Process 3.0: Incident & Alert Management

- Input: Detection events
- Output: Alerts to operator, dispatch to responders
- Data Stores: Incident (D2), Alert (D3)

Process 4.0: Operator Interface

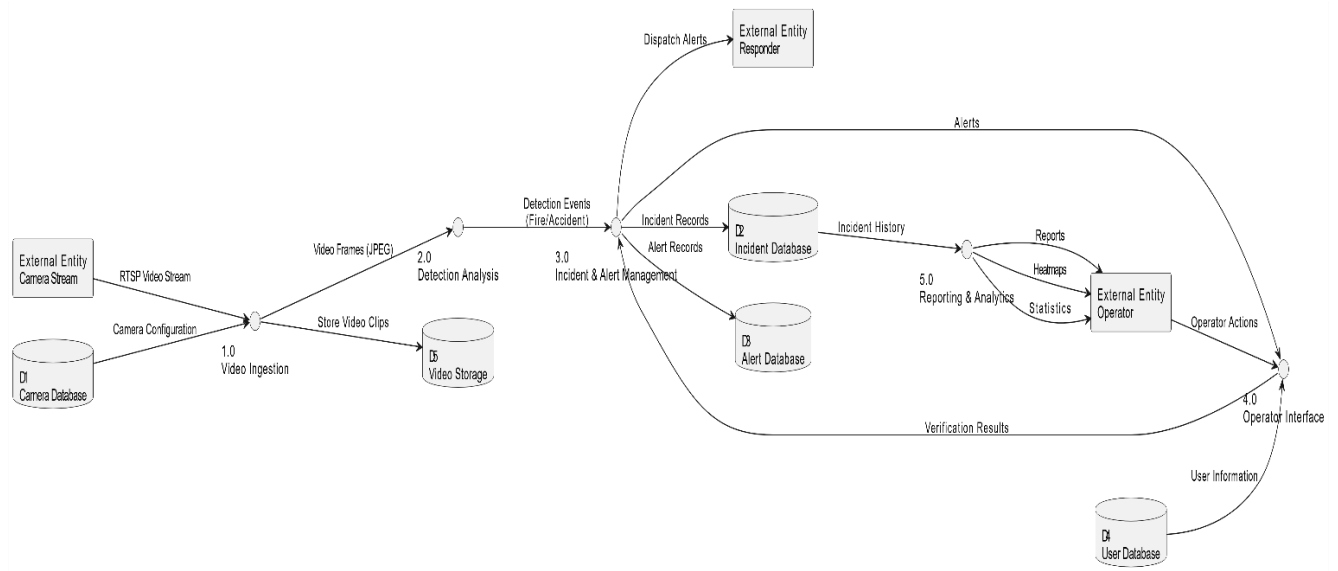
- Input: Alerts, operator actions
- Output: Verification results
- Data Stores: User (D4)

Process 5.0: Reporting & Analytics

- Input: Incident history
- Output: Reports, heatmaps, statistics
- Data Stores: Incident (D2)

Data Stores:

- D1: Camera Database
- D2: Incident Database
- D3: Alert Database
- D4: User Database
- D5: Video Storage (file system or S3)



11.3 Level 2 – Detailed DFD for Detection Process (Process 2.0)

Process 2.1: Frame Preprocessing

- Input: Raw frame
- Output: Resized normalized frame
- Details: Resize to 640x640, normalize pixel values (0-1), convert to tensor

Process 2.2: Fire Detection Inference

- Input: Tensor
- Output: Fire confidence score, bounding boxes
- Details: Runs YOLOv8-based fire model

Process 2.3: Accident Detection Inference

- Input: Tensor (sequence of 5 frames)
- Output: Accident confidence, type (collision/rollover)

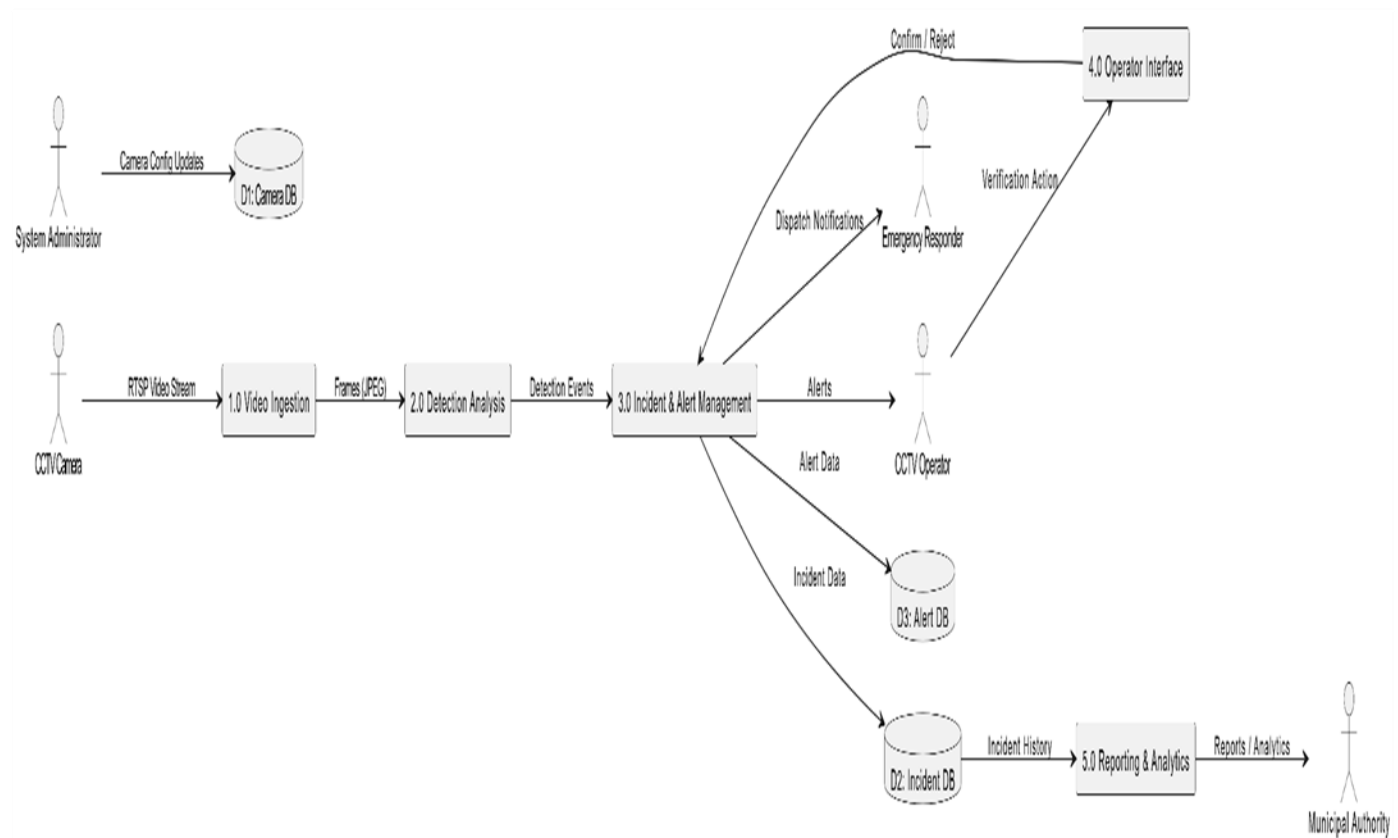
- Details: Uses temporal CNN + object tracking

Process 2.4: Severity Classification

- Input: Detection results
- Output: Severity level (enum)
- Details: Flame area % for fire; speed + vehicle count for accident

Process 2.5: Temporal Validation

- Input: Detection results from multiple frames
- Output: Confirmed detection (true/false)
- Details: Require 3 out of 5 consecutive frames positive



12. Sequence Diagrams

12.1 Fire Detection and Alert Sequence

Actors & Objects: Camera, VideoProcessor, DetectionEngine, Database, AlertService, OperatorDashboard, EmergencyServiceAPI

Sequence:

Camera -> VideoProcessor: streamFrame()

VideoProcessor -> VideoProcessor: bufferFrame()

VideoProcessor -> DetectionEngine: analyzeFrame(frame)

DetectionEngine -> DetectionEngine: runFireModel()

DetectionEngine -> Database: createIncident(type=Fire, severity, cameraId)

Database --> DetectionEngine: incidentCreated(incidentId)

DetectionEngine -> AlertService: triggerAlert(incidentId)

AlertService -> Database: getIncidentDetails(incidentId)

AlertService -> OperatorDashboard: pushAlert(alertData)

OperatorDashboard -> OperatorDashboard: playSound(), showPopup()

OperatorDashboard -> Operator: displayAlert()

Operator -> OperatorDashboard: clickConfirm()

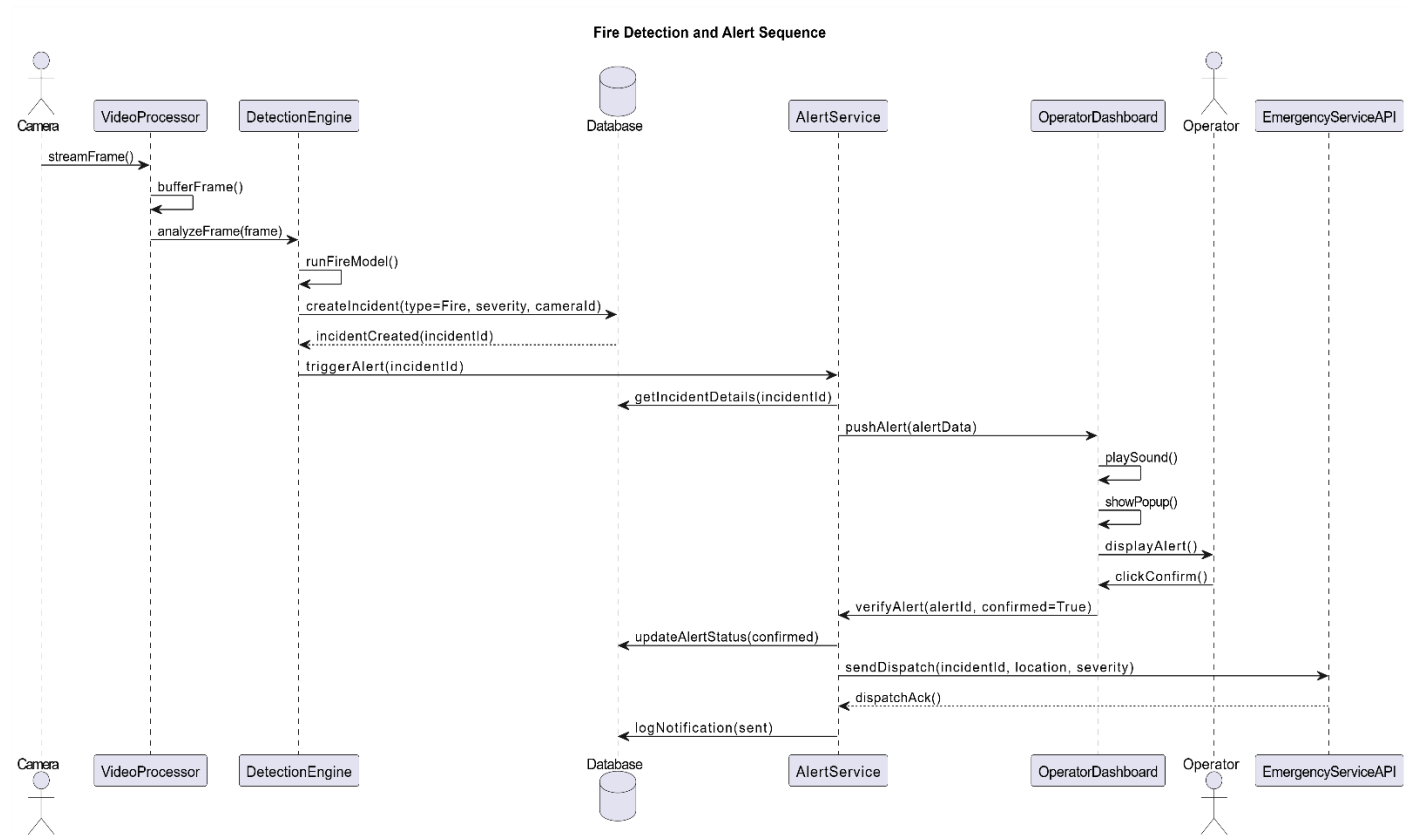
OperatorDashboard -> AlertService: verifyAlert(alertId, confirmed=True)

AlertService -> Database: updateAlertStatus(confirmed)

AlertService -> EmergencyServiceAPI: sendDispatch(incidentId, location, severity)

EmergencyServiceAPI --> AlertService: dispatchAck()

AlertService -> Database: logNotification(sent)



12.2 Accident Detection and Dispatch Sequence

Actors & Objects: Camera, AEngine, Database, NotificationService, AmbulanceAPI, PoliceAPI

Sequence:

Camera -> AEngine: videoStream()

AEngine -> AEngine: objectDetection()

AEngine -> AEngine: opticalFlow()

AEngine -> AEngine: detectCollision()

AEngine -> Database: createAccidentRecord(type, severity, GPS)

Database --> AEngine: recordCreated()

AIEngine -> NotificationService: requestDispatch(incidentId)

NotificationService -> Database: getIncidentDetails(incidentId)

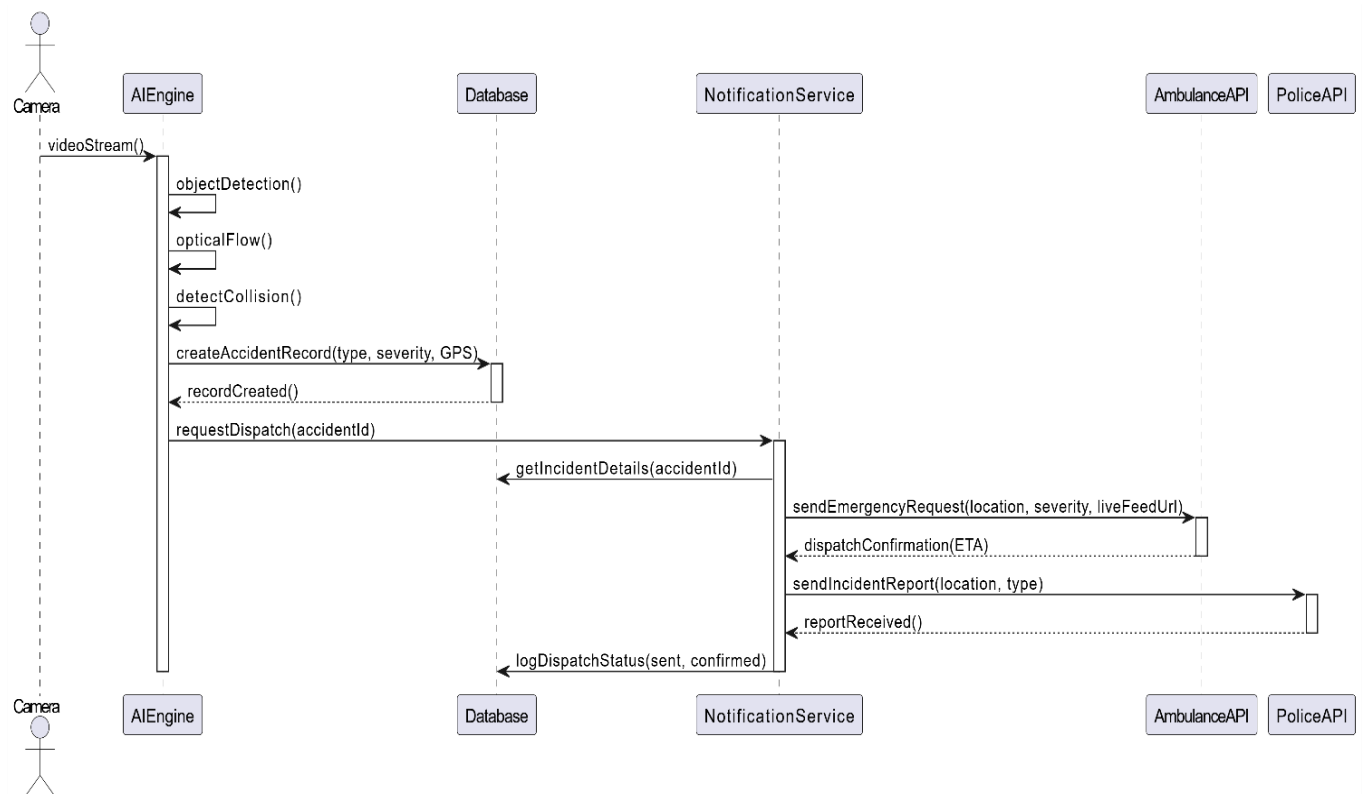
NotificationService -> AmbulanceAPI: sendEmergencyRequest(location, severity, liveFeedUrl)

AmbulanceAPI --> NotificationService: dispatchConfirmation(ETA)

NotificationService -> PoliceAPI: sendIncidentReport(location, type)

PoliceAPI --> NotificationService: reportReceived()

NotificationService -> Database: logDispatchStatus(sent, confirmed)



12.3 Operator Verification Sequence

Actors & Objects: Operator, Dashboard, AlertService, Database

Sequence:

Operator -> Dashboard: viewPendingAlerts()

Dashboard -> AlertService: getUnverifiedAlerts()

AlertService -> Database: queryAlerts(status='Pending')

Database --> AlertService: returnAlertList()

AlertService --> Dashboard: displayAlertList()

Operator -> Dashboard: selectAlert(alertId)

Dashboard -> Database: fetchIncidentDetails(alertId)

Database --> Dashboard: incidentData

Dashboard -> Operator: displayVideoClip(), location, severity

Operator -> Dashboard: clickConfirm() OR clickFalseAlarm()

Dashboard -> AlertService: submitVerification(alertId, decision, notes)

AlertService -> Database: updateAlertStatus(decision, notes)

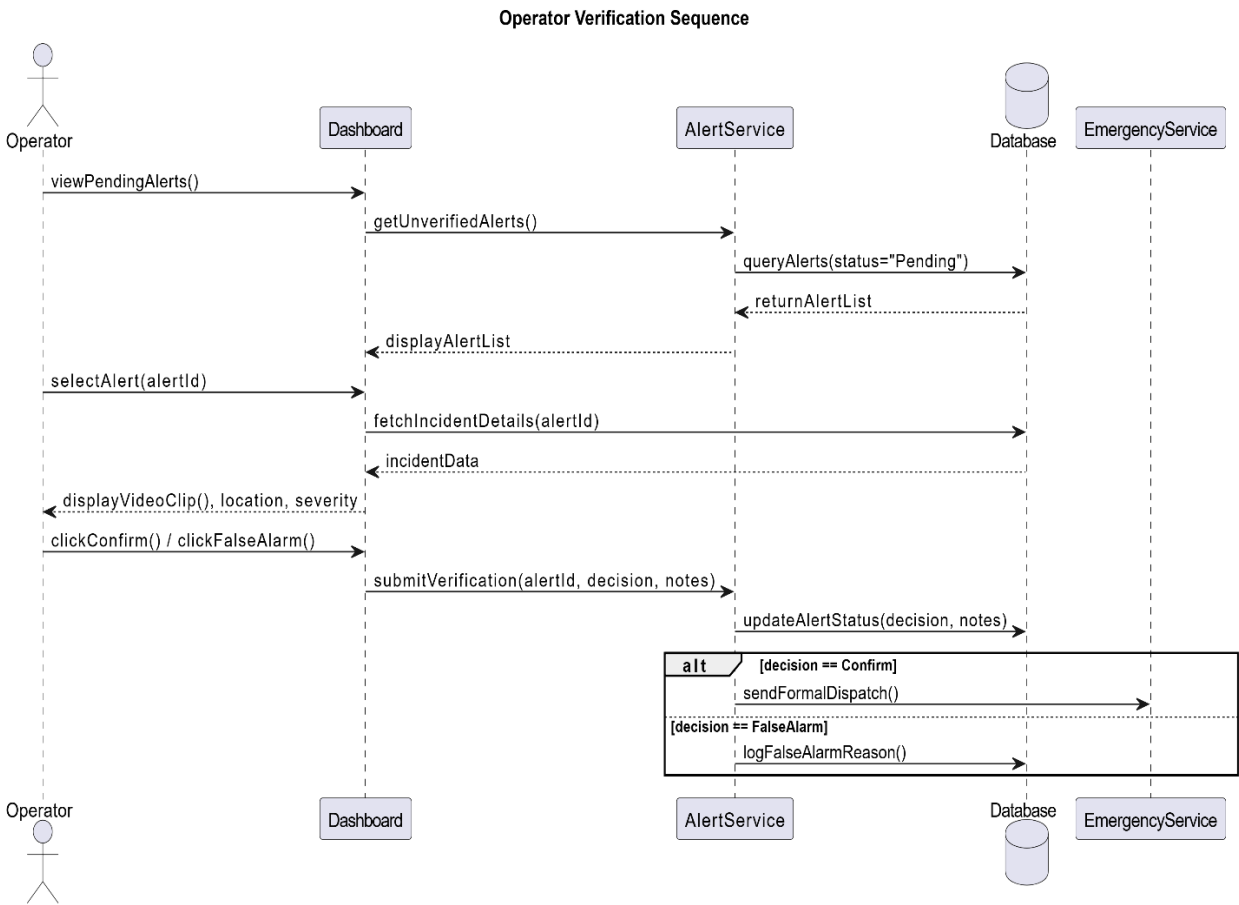
alt [decision == Confirm]

 AlertService -> EmergencyService: sendFormalDispatch()

else [decision == FalseAlarm]

 AlertService -> Database: logFalseAlarmReason()

End



12.4 Admin Camera Registration Sequence

Actors & Objects: Admin, AdminUI, CameraManager, StreamTester, Database

Sequence:

Admin -> AdminUI: enterCameraDetails(name, location, GPS, streamUrl)

AdminUI -> StreamTester: testStream(streamUrl)

StreamTester -> Camera: attemptConnection()

Camera --> StreamTester: connectionSuccess()

StreamTester --> AdminUI: previewAvailable()

AdminUI -> Admin: showLivePreview()

Admin -> AdminUI: drawPrivacyMasks()

AdminUI -> AdminUI: storeMaskCoordinates()

Admin -> AdminUI: selectSensitivity(Medium)

Admin -> AdminUI: clickRegister()

AdminUI -> CameraManager: registerCamera(details)

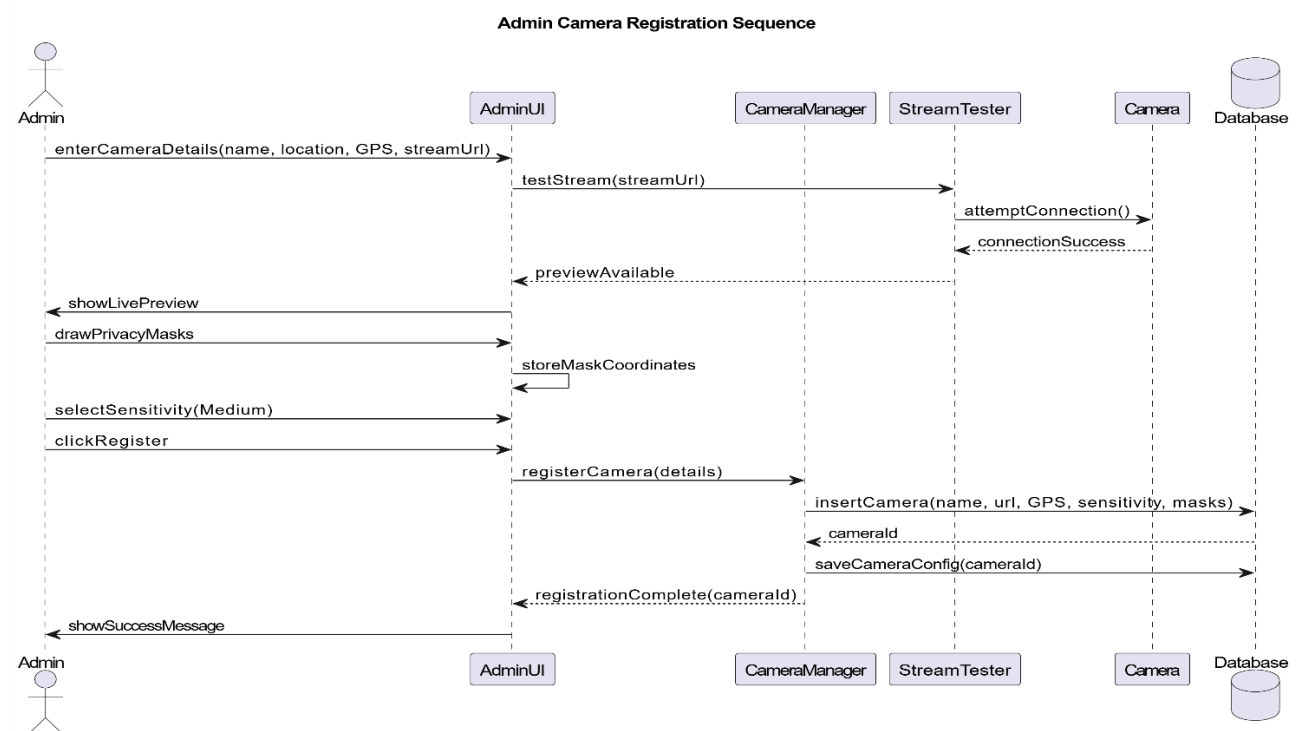
CameraManager -> Database: insertCamera(name, url, GPS, sensitivity, masks)

Database --> CameraManager: cameraId

CameraManager -> Database: saveCameraConfig(cameraId)

CameraManager --> AdminUI: registrationComplete(cameraId)

AdminUI -> Admin: showSuccessMessage()



13. Non-Functional Requirements

NFR-01 : Detection latency (from incident to alert)

NFR-02 : Dashboard alert display time

NFR-03 : Fire detection precision (true positive rate)

NFR-04: Fire detection false positive rate

NFR-05 : Accident detection precision

NFR-06 : System uptime

NFR-7 : Detailed logs for all detection events, user actions, system errors |

NFR-8: False alarm handling

NFR-9: Data protection

14. Constraints & Assumptions

14.1 Technical Constraints

Technical constraints are limitations imposed by hardware, software, network, or platform choices.

TC-01 : Camera Stream Protocol –

The system only supports RTSP, H.264, and H.265 streams over TCP/IP. Legacy analog cameras (BNC) require encoders. Older CCTV systems may need additional hardware upgrades.

TC-02 : Minimum Camera Resolution -

Detection models require a minimum resolution of 640x480 pixels. Lower resolution results in reduced accuracy. Cameras below this resolution must be replaced or upgraded.

TC-03 : Network Bandwidth –

Each camera requires at least 2 Mbps upload bandwidth for real-time streaming. For 100 cameras, minimum 200 Mbps dedicated upload. Municipal networks may need upgrading; cellular backhaul may be insufficient.

14.3 Operational Constraints

Operational constraints relate to how the system will be used, managed, and maintained in production.

OC-01 : 24/7 Operator Availability –

The system requires at least one trained CCTV operator logged in during all operational hours (24/7) to verify alerts. Shift scheduling must be maintained. Unmanned periods will cause missed escalations.

OC-02 : False Alarm Handling Process –

Operators must follow a standard procedure for marking false alarms, including selecting a reason from a dropdown. Requires operator training and compliance.

14.5 Assumptions

Assumptions are conditions that the project team believes to be true but are not guaranteed. If any assumption is false, the system may not meet expectations.

AS-01 : Existing CCTV Infrastructure –

The city already has CCTV cameras installed at key locations (intersections, public squares, commercial areas). High – New camera installation would significantly increase budget and timeline.

AS-02: No Malicious Use –

The system will not be deliberately fooled (e.g., playing fire videos to trigger false alerts) or hacked. Low – Basic security measures (authentication, input validation) are assumed sufficient for initial deployment.

AS-03: Legal Approval –

Municipal authorities have obtained all necessary legal approvals for automated surveillance and data retention. | High – Without approvals, the system cannot be deployed.

15. Conclusion

Summary of the System

The ***CCTV-Based Emergency & Rescue Detection System*** presented in this Software Requirements Specification addresses a critical real-world problem: delayed emergency response due to limitations of human CCTV monitoring. By automating the detection of : ***fire incidents*** and ***road accidents*** using artificial intelligence and computer vision, the system reduces the time between an incident occurring and emergency services being notified from minutes to seconds.

The system is designed as a web-based platform with four primary user roles:

CCTV Operators: who receive and verify alerts

Emergency Responders: who receive dispatch notifications with live video and GPS

System Administrators: who manage cameras, users, and configuration

Municipal Authorities: who access analytics and reports

Key features include real-time video ingestion, AI-based detection (fire and accident), severity classification, automatic video clip recording, operator verification workflow, push notifications to emergency services, incident heatmaps, response time analytics, and comprehensive reporting.

The ***CCTV-Based Emergency & Rescue Detection System*** is not merely a software project – it is a public safety infrastructure investment. By transforming passive surveillance into active emergency detection, the system has the potential to save lives, reduce property damage, and build more resilient communities. This SRS document provides a complete, unambiguous, and stakeholder-approved foundation for the design and development phases. With continued collaboration between the development team, municipal authorities, and emergency services, the system can move from specification to reality, making cities safer one camera at a time.